EXCALIBUR-Medic: Can Distillation and Retrieval Enable On-Device Clinical Inference on 2 GB Edge Devices?

Sandy MacDonald

A dissertation submitted in partial fulfilment
of the requirements for the degree of
Master of Science in Artificial Intelligence
of the
University of Aberdeen.



Department of Computing Science

August 18, 2026

Declaration

No portion of the work contained in this document has been submitted in support of an application for a degree or qualification of this or any other university or other institution of learning. All verbatim extracts have been distinguished by quotation marks, and all sources of information have been specifically acknowledged.

Signed: Sandy MacDonald

Date: August 18, 2026

Abstract

Unlike state-of-the-art large language models, small language models deployed on constrained edge AI devices often struggle with reliable medical literature recall. Take for instance, when asked about the thyroid disease of 'hypothyroidism', the 1 billion parameter model at the centre of this work listed erroneously 'weight loss' among the retrieved symptoms. Getting the textbook discriminator between a underactive and overactive thyroid, a complete inversion from the truth and which the model worryingly presented and stated with complete confidence in it's answering. To address such regressions without opting for higher parameter sized models that the hardware constraints cannot handle or internet connectivity, this work presents a hybrid method; the 1B student model was trained to imitate an 8B teacher medical fine-tuned model (knowledge distillation), compressed from 2.4 GB to ~771 MB, and paired with a classical keyword search model across 19,411 medical documents drawn from the NIH published clinical question-answering sets, the MedlinePlus health topics corpus and the US FDA drug labels. The resulting output model runs and infers entirely offline and on device, on a 2 GB Raspberry Pi 5 which currently retails at around 60 Pound Sterling. Comprehensive benchmarking of the resultant model, named EXCALIBUR-medic shows the systems viability for on-device edge AI deployment. It's memory footprint peaks at 1,638 MB resident yet leaves 929 MB out of the total 2,010 MB Linux addressed memory of the Pi 5, since GGUF exported model file is memory-mapped; it generates about 15 tokens or roughly 11 words per secound. This performance does not throttle the edge device and it draws a comfortable 5.4W under sustained load, and it holds it clock speeds across two independent 40-minute stress tests. Retrieval raises the proportion of source-traceable medical content from 0.31 to 0.71, against a oracle ceiling of 0.72 for perfect clinical retrieval. The aforementioned evaluation proves and reveals that the two main methods, knowledge distillation and retrieval serve distinct functions. The RAG powered retrieval supplies the factual accuracy of the model; with the same model that previously inverted the hypothyroid symptoms now answers that particular prompt with the correct reasoning and answering when a reference is place in front of it, so imitation (knowledge distillation) alone never fixed recall. That distilled training is instead almost entirely responsible for shaping the models output into a usable, checkable and verifiable structure, which improves from a baseline of 0% to a near perfect 97.5%. Distillation also taught the EXCALIBUR-Medic model integral clinical safety behaviours, such as refusing all 16 unanswerable held-out questions with no false refusals across the 40 answerable ones, copying critical make or break dosage and warning text verbatim rather than paraphrasing or potentially hallucinating it. As reported evaluation metrics reflect textual overlap and accuracy to the source texts rather than actual formal medical correctness, clinical validation remains necessary and thus the model

was instructed to determine medical questions from unrelated user questioning and provide robust guards when necessary.

Acknowledgements

I would firstly like to thank The Data Labs Academy for funding my studies and providing a robust scholarship programme that greatly aided me during my studies. Furthermore, I would like to acknowledge and thank my Project Supervisor, Dr Dewei Yi for his comprehensive support, domain knowledge and expertise, and who provided very helpful suggestions and recommendations throughout the duration and curation of my Project.

Contents

1 Introduction **10**

1.1 State of affairs 10
1.2 Motivation 11
1.3 Objectives 14
1.4 Outline 15

2 Background **17**

2.1 What clinical model benchmarks actually show, and what they do not 17
2.2 Small language models and their limits of parametric factual clinical recall 18
2.3 Model compression techniques; distillation, fine-tuning and quantisation 20
2.4 Grounding medical query answering generation in retrieved clinical corpus text . 22
2.5 Safeguards for interactive generative clinical questioning and answering systems 24
2.6 Background summary 26

3 Literature Survey **27**

3.1 Medical focused large language models 27
3.2 Retrieval augmented powered clinical questioning and answering 28
3.3 On-device small language model edge deployment 29
3.4 The gap identified from the surveyed literature 30

4 Design **31**

4.1 The EXCALIBUR-Medic model pipeline overview 31
4.2 Training the distilled excalibur model 33
4.3 Constructing and assembling the clinical retrieval corpus 35
4.4 Ranking the retrieved medical corpus documents 37
4.5 The retrievers decision layer 39
4.6 The verbatim path 41
4.7 Design summary 42

5 Implementation **44**

5.1 The three main implementing artefacts of EXCALIBUR-Medic 44
5.2 Building, training and exporting the EXCALIBUR-Medic model 45
5.3 The deployed edge runtime inference system for EXCALIBUR-Medic 47
5.4 The EXCALIBUR-Medic Model edge benchmarking process 50

6 Evaluation **52**
6.1 Model distillation and retrieval notebook results 52
6.2 Raspberry pi 5 on-device deployment measurements 55

7 Discussion **57**

8 Conclusion **59**
8.1 Limitations & Future work 59

A Appendix A: Source code, reproduction artefacts and user manual **61**
A.1 GitHub Repository for Excalibur 61
A.2 Optional excalibur training artefacts 61
A.3 Setting up the training environment 62
A.4 Building the excalibur model 62
A.5 Deploying to the Pi 5 62
A.6 Inference and asking questions on the Pi 5 62
A.7 Benchmarking on the Pi 5 63
A.8 Video demonstration of EXCALIBUR-Medic 63

List of Tables

1.1 Arguments for deploying the model both on device and offline. 13
1.2 Motivating factors for choosing the Pi 5 2 GB as the baseline edge AI device target. 14

2.1 The three compression operations applied in this work. 22
2.2 Dense and sparse retrieval compared against the criteria that matter for on-device clinical retrieval. 24
2.3 Three safeguards, and the point in the process at which each acts. 25
2.4 The chain of argument developed in this chapter, and what each step rules out. . . 26

3.1 Key surveyed literature findings against the strict target constraints of the Pi 5 2 GB deployment target edge device. 30

4.1 The three corpus sources. 36
4.2 The four weighted adjustments to the retrievers ranking. 39
4.3 The five guards, in firing order. Each exists because of an observed failure; thresholds are given in Chapter 5. 40
4.4 Summary of how the system determines when a decision is taken. 43

5.1 The three major implementation artefacts and their purpose, plus whether or not the artefact is deployed on-device on the Pi 5 edge target. 44
5.2 Everything required on the Pi 5 edge device for interactive questioning and answering inference. 50

6.1 Compute and memory measurements on the Raspberry Pi 5 using benchmark_pi.sh (2 GB, headless, Raspberry Pi OS Lite). 55
6.2 Power and thermal behaviour, two independent runs measured from benchmark_pi.sh –energy. 56

A.1 Optional reproduction artefacts and the stage each removes. 61

List of Figures

1.1 Compiled MedQA Reported Model(s) Accuracy, 2021-2024 11
1.2 OpenAI's ChatGPT Pricing Plans Range 13
1.3 The Pi 5 currently retails at a still affordable and in stock £60-ish pound sterling as of August 2026 PiHut (2026) 14

2.1 A collection of the most downloaded small language models on hugging face, note the downloads in the , showing clear interest and demand for SLM's Ji (2026) . . 19
2.2 Sparse vs dense retrieval Sumit (2023) 23

4.1 EXCALIBUR-Medic Pipeline, showing the interaction of both retrieval and distillation of model, on how a question answering is actually reached and pipelined, note the shared responsibility for answering. 32
4.2 An abstract view of the distillation process before the quality gate, note the reasoning and answering pairs behaviour distilled down from the medical teacher model onto the student model. 34
4.3 The distillation pipeline of EXCALIBUR-Medic 35
4.4 How the three input source corpus are assembled into the master single shared schema retrieval corpus. 36
4.5 How a user question is normalised and ranked through the retriever itself. 38
4.6 The facet guard enforces aspects must be defined in the retrieved source corpus documents 41
4.7 The type of questions that are handled verbatim. 42
4.8 A worked query through the EXCALIBUR-Medic design ordered component pipeline 43

6.1 The one billion parameter student model capacity ceiling 52
6.2 Retrieval ladder of the model, showing how each tuning adjustments added to the overall retrieval accuracy on the 1,200 held out MedQuAD question pairs. 53
6.3 Supervised Fine Tuning (SFT) training loss and validation perplexity 53
6.4 Base student model versus distilled student model grounding and inference compliance, showing that distillation does indeed improve over the base model. . . . 54
6.5 Grounding ladder, what it is worth and the shipped adapted model against it's own floors 54
6.6 Interactive inline per query stats 56

Chapter 1

Introduction

1.1 State of affairs

The use of applying large language models to the domain of clinical questioning and answering has rapidly evolved in the past couple of years. Using an industry standard medical questioning and answering benchmark known as MedQA and which is comprised from many official medical board examinations and shows how closely models match human expertise clinical knowledge, it can be seen that in 2021 one year before the launch of OpenAI's ChatGPT, the best reported model baseline only answered around 36.7% of English questions correctly. Jin et al. (2020) Just two years later in 2023, Google's Flan-PaLM model reached 67.6% accuracy, this was followed in the same year by OpenAI's GPT 4 which scored 86.5% on the same benchmark that just two years earlier models could barely pass the random chance rate of 25%. Singhal et al. (2023)Nori et al. (2023) This rapid and clear progression not only shows that modern language models can now easily exceed the threshold to pass human examinations, but are also now well suited to the acceleration of clinical answering and questioning tasks as evidenced by how well they perform when exposed to such a medical domain. As evidenced by the MedQA accuracy timeline figure below 1.1, while the benchmark was previously dominated by proprietary closed weight models, open weight models have also caught up and are equally just as good at clinical domain questioning and answering, with that particular open weighted HuatuoGPT-o1-70B model being some 70 billion parameters, around seventy times more than the baseline Pi 5 2 GB device can hold in memory. This thesis asks the question, what survives such a reduction and compression; can distillation carry the behaviour of a much larger medical model into one so small it can fit and deploy comfortably into a 2 GB Pi 5 device and whether retrieval can supply back the knowledge that no longer fits inside it.

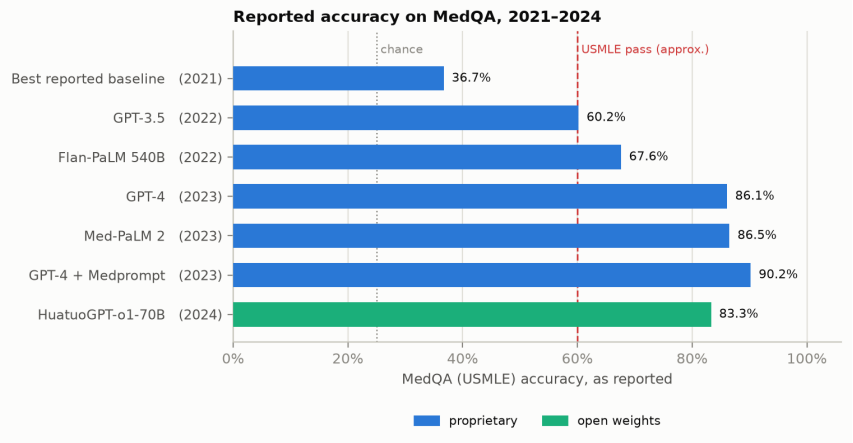


Figure 1.1: Compiled MedQA Reported Model(s) Accuracy, 2021-2024

1.2 Motivation

Throughout healthcare, clinical questions from patients constantly come-up although most of these questions go unanswered by the clinician. Take for instance the following literature, a clinically compiled review of 7,012 questions recorded just after a patients visit found that these questions are prompted at a rate of around 0.57 per patient visit, and that only 51% of these patient questions were actually followed up on by the clinician and that only a answer was given to 78% of the aforementioned pursued questions show that roughly three in every five patient question was abandoned or unresolved by the clinicians. Del Fiol et al. (2014) The two main discerning factors for this poor resolution rate was a lack of doubt that a useful answer to the patients question actually existed and lack of time on the clinicians behalf. This shows that the main problem is not a gap in the actual medical literature but both are failures of access. The questions that were not resolved the most in the compiled review concerned answers pertaining to drug treatment at 34 percent of answered questioning, i.e. dosage amounts, brand and product labelling existed verbatim in many freely accessible medical corpora however with the aforementioned bottleneck of access, they were not followed up on.

To resolve the bottleneck of access, language models can be used and are well placed to alleviate such a human bottleneck. Unanswered clinical queries and questions could be given to the language models as a input, where the patients question is answered from indexed medical reference materials in secounds via safe-guarded model output rather than wait on a human answer thereby eliminating the time cost causing half of these unanswered questions to be abandoned, furthermore the model can determine whether or not a answer actually exists or not rather resolving the uncertainty that also causes questions to go unresolved.

Where capacity is at it's thinnest, the burden and bottleneck of *access* is worse, and even more clinical questions go unanswered. For instance worldwide, in 2025, there are still some 2.2 billion people still not connected to the internet, with 96% of them living in impoverished countries, of which is further complicated by a urban rural divide, with rural internet access at 58% compared to a urban connectivity rate of 85%. ITU (2025) These are also the exact demographics in which the density of clinicians is lowest. Organisation mondiale de la santé (2016) The models shown

in Figure 1.1 cannot be used to serve them. As every one of these models is run using someone else's expensive and unobtainable hardware and servers in these particular demographics. Rather, the inaccessible deployment strategy of calling said third party hosted language models there is a different way of allowing the access needed to answer these unanswered clinical questions. Instead of a deployment that calls a model, there exists a deployment strategy instead that holds the model with the deployment, i.e. the model is offline and on device, bypassing the third-parties all-together. This deployment rather than call someone else's model hosted elsewhere, holds the model that will serve them and answer their clinical questions. Four clear arguments justify why such an approach could be beneficial.

The first being that a deployed model must be reached where a internet access can not be assumed, such as in field hospitals, urban ambulance care or remote disaster response, in such scenarios the models end-user internet access could be constrained, non-existent or in flux, thus such a model must work offline precisely when it is needed most clinically rather than experience model API calling failure exhibited in models that are called and not held on device. Keeping the model on device and offline ensures that clinical questioning inference can always be achieved so long as the device itself can be reached thus cutting out the additional network fail state. Bypassing the network also means reducing latency as their exists no network in the way to stream what the model produces, thus the on device model is not waiting to connect to a server, but generates tokens itself and thus the only latency it is exposed to is it's own.

Secondly, models that are called have to be sent the users prompt query, this is complicated as health information is regulated as a special category of personal data under both the UK and EU GDPPR, where such queries can only be legally processable under strict and narrow conditions. EU (2016) A query called to the model need not even carry a patient's legal name to qualify to the strict data regulation, even locations, time of query or even a patients specific condition can de-anonymise, expose and identify a patient. Thus transmitting it to a called model hosted elsewhere from the clinical premise triggers data processor obligations, impact assessments risks and if the origin location of the model sits outside the United Kingdom or the European Economic Area, a rigorous transfer legal mechanism brings with it intensified legal risks and burdensome and time-consuming contractual clauses that could bring hefty fines among other legal punishments if patient data is not protected nor respected in the transmission from query to model. ECJ (2020) However, a model that is deployed and held locally while not fully mitigating the GDPPR medical requirements, it bypasses and removes the aforementioned transmission obligations and simplifies and streamlines and reduces legal work and upkeep and allows the end user to divert more time and attention that would have otherwise been spent on transmission legal work.

Thirdly, a remote called model has no inherit fixed identity. Hosted online models can be recalled, removed, withdrawn without even a notice to the end user. Furthermore, deploying a model locally allows the actual makeup of the model to be seen and inspected and validated against whereas called models are usually closed weight and such information is irretrievable and hidden. While regulation is arriving to prevent such interruptions, such as the landmark EU Artificial Intelligence Act, it is currently not yet implemented and thus model providers have the power to revoke access to their models at will, which in the case of clinical inference brings uncertainty and potential

harm that such a local deployed held model does not as the end user has full control of their own on device hardware running the clinical questioning and answering inference whereas relying on a third-party model, the end-user has no control and control and access to the model could even be revoked. EU (2024) A deployed model is held deterministically on device, a end-user can physically hold the device, is identifiable by a cryptographic digest and completely inspect-able and traceable in full, is far more maintainable and tractable than a third party called model which weights are closed and easily changeable.

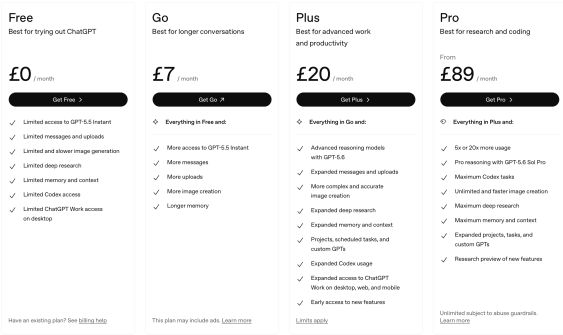


Figure 1.2: OpenAI's ChatGPT Pricing Plans Range

Fourthly and lastly, a hosted model that is called via API and served elsewhere by third-parties is usually billed per query call. That means costs associated with the model scale linearly and potentially without limit. These costs and pricing have become so substantial enough that researchers have devised token cascading optimisation systems that aim to reduce cost whilst maintaining model output quality. Chen et al. (2023a) Whereas a local on device model can be deployed and targetted to a hardware which is bought once, at a few watts whereby drawing orders of magnitude less watts than the servers hosting the called models, where queries to the local model carry no marginal cost per user clinical prompt query at all. Plus unlike the hosted models, carrying no dependency, the model will not be repriced, nor rate-limited nor discontinued as the user controls the deployment hardware and has complete control. Costs thereby could rapidly spiral out of control if using hosted third-party models whereas hosted locally and on device it really cannot.

Table 1.1: Arguments for deploying the model both on device and offline.

Argument	**A model that is called**	**A model that is held**
Availability	Unavailable whenever the network is.	Runs with no connection.
Data protection	The query must be transmitted; health data engages processor obligations and cross-border transfer rules.	No query leaves the device, so no transfer arises.
Validation	Weights may change without notice, invalidating any prior evaluation.	A fixed model file and inspect-able.
Cost	Billed per query; scales without limit.	Bought once; no marginal cost per query.

The statement of a design premise that ensures that the model is held locally and on edge AI devices not as a limitation but as a potential answer to the access bottleneck identified with clinical answering.

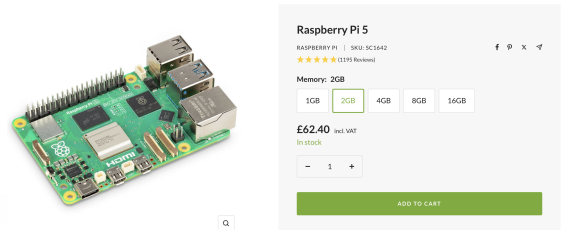


Figure 1.3: The Pi 5 currently retails at a still affordable and in stock £60-ish pound sterling as of August 2026 PiHut (2026)

The 2 GB Pi 5 Edge IoT Device was selected as the baseline target for the on device stratagem as it is representative rather than uniquely suitable as it is inexpensive, draws single digits watts, needs no specialist cooling solution, and is already established both as a class and as a platform for low cost clinical deployments in hospitals and general practitioners worldwide. SONIA (2020) Mehta (2025) The 2 GB memory target variant of the Pi 5 was chosen deliberately as the floor of IoT memory, as it is a popular memory choice of IoT devices not just the Pi 5 family and if the model methodology works at the 2 GB floor it will comfortably hold on any memory sizing above it.

Table 1.2: Motivating factors for choosing the Pi 5 2 GB as the baseline edge AI device target.

Requirement	**Raspberry Pi 5 (2 GB)**
Cost	One-off purchase at commodity price, with no per-query charge and no subscription.
Power	Draws single-digit watts, so battery and solar operation are practical where mains supply is unreliable.
Thermal envelope	Requires no specialist cooling or controlled environment.
Availability	Mass-produced, globally distributed and supported, rather than specialist hardware requiring an import path.
Clinical precedent	Single-board computers of this class are already used for low-cost point-of-care instrumentation in low-resource settings.
Memory ceiling	The 2 GB variant is the floor rather than the comfortable case: a method that holds here holds on any larger device.

1.3 Objectives

The aim of this thesis is to investigate whether a language model small enough to run on device and offline on a 2 GB Pi 5 Edge Device can serve as a reliable and trustful interactive clinical questioning and answering reference, of which users will be able to interactively query and prompt the EXCALIBUR-Medic model their clinical and medical queries and get a reliable and factual clinical response answer, through intensive use of knowledge distillation and retrieval primarily, and to determine, establish, and report what each component of such a model pipeline contributes to that medical answer reliability and precision.

The six objectives are as follows:

1. **Assemble an offline clinical and medical corpus**, comprised of clinical questioning and answering material, drug product labelling and general consumer health topics, into one indexed corpus covering a range of medical conditions, drugs and general health questioning, measuring both what it can and cannot answer.

2. **Distil a student 1B model from an 8B medical teacher**, generate chain of thought supervision prompts from a quantised medical fine-tuned teacher, filter the teacher against explicit quality criteria from the corpus and fine-tune the student EXCALIBUR-Medic model on what survives the quality check.

3. **Compress the distilled student model to fit onto the 2 GB edge AI device**, fusing, model conversion and quantisation of the trained student model to a compressed model artefact that runs within a 2 GB device alongside the assembled index, without much measured loss from the baseline non quantised model.

4. **Build a interactive retrieval inference environment under the strict hardware requirements**, an offline retriever without an embedding model and no runtime dependencies, plus tune said retriever against held out questions to determine it's true non-assumptive performance, a interactive chat style retriever that a end user can query medical questions and the model returns a question and reasoning for said returned question.

5. **Implement robust medical inference guards**, to refuse in the interactive model when a retrieval has no sourced answer from the corpus, reject too broad of clinical questions, and provide verbatim corpus quotation where a model generation is unsafe, e.g. drug dosages.

6. **Benchmark and evaluate the deployed EXCALIBUR-Medic model on the target edge device hardware**, measure answer grounding to the assembled corpus, format compliance of the model, retrieval accuracy, model throughput, memory utilisation, thermal behaviour on the edge device itself.

1.4 Outline

- **chapter 1: Introduction** - Presents the domain of the research topic, motivation and objectives of the research plus an outline of all the chapters included in the dissertation.

- **chapter 2: Background** - Concerns the background abstract knowledge that are foundational to the work. What current medical benchmark models show and what they omit, why a small language of this size cannot hold the corpus clinical domain in it's parameters alone, the compression techniques used in fitting the model onto the Pi 5 device and safeguarding mechanisms that compensate the retrieval component of EXCALIBUR-Medic.

- **chapter 3: Literature Survey** - Surveys existing literature, existing large medical models tuned to a medical domain, how retrieval corpus are composed, rag aided clinical inference systems and existing on device deployment work, and from these identifies gap in the current existing research that this work attempts to address .

- **chapter 4: Design** - Sets out and defines the architecture of this work, how the distillation process will work, what sources the medical retrieval index is composed of, what ranking adjustments to the documents will be carried out and the corresponding and judging five safeguards which determine whether a user medical query is answered or not.

- **chapter 5: Implementation** - Describes the three main implementing artefacts in the work, the juypter notebook that builds, trains, compresses and exports the model, the inference

script that is deployed to the Pi 5 to allow interactive medical questioning and answering, and the benchmarks that measure such a deployment .

- **chapter 6: Evaluation** - Reports back using figures and benchmark returned data, how well the system performs and which component contributes to what in regards to both the measured cost on the Pi 5 hardware, the compute, memory, power and whether or not the model causes the Pi 5 to thermal throttle or not.
- **chapter 7: Discussion** - Further commentary on the evaluation data, what has been established by the system and what hasn't.
- **chapter 8: Conclusion** - Ends the work with what has been contributed, what the limitations are and what future work is still needing to be done, including a conversational context that remembers the past query, a non-US specific biased training corpus and a cross-platform training runtime instead of the MLX one currently.

Chapter 2

Background

2.1 What clinical model benchmarks actually show, and what they do not

The timeline and rapid large language model progression in regards to medical knowledge capabilities seen in Figure 1.1 is very real. These huge large language models are genuinely capable at questioning and answering in a clinical domain. Although, the actual measured progression is narrower than the headline figures as there are some core interesting reasoning caveats associated with those benchmarks that influenced how the architecture of EXCALIBUR-Medic was adopted.

For example, that particular benchmark was derived from 3 main clinical sources, all of which share a similar reported structure. The MedQA dataset is a multiple choice questionnaire examination compiled by the United States Medical Licensing Examination, which contains a test split of 1,273 questions. Jin et al. (2020) Similarity, the Indian MedMCQA was compiled of likewise Indian medical entrance examination multiple choice questions and contained some 21 different clinical subject matters across over 6,100 item test pairs. Pal et al. (2022) Finally, the PubMedQA differs slightly in form but not the kind of benchmarked data, each clinical question is accompanied by a abstract from which the correct clinical answer may be derived from, and the model can either select from a yes, maybe or no answer. Jin et al. (2019) It is clear then, from all 3 referenced benchmarks, and in every case of when the model is asked at runtime, both the answer and the source text containing it is placed before the model before they can even ask and answer the question.

From closer inspection of how these language models achieved success in the domain of clinical inference questioning and answering, the following three design decisions were taken for the betterment of EXCALIBUR-Medic clinical factual recall acknowledging said benchmark caveats.

Model failure in the benchmarks, is shown to be confident in it's incorrect answers, as language models can and will generate text that it deems locally plausible without respect if the actual incorrect answer is factually supported or not, a behaviour known as model hallucination. Ji et al. (2023) This is clearly a problem if a user prompts a model for a symptom of a specific disease or a drug dosage and the answer the model generates is completely and utterly false and has no factual basis, this could harm the end user of the model and thus hallucinations must be, specially for clinical purposes be minimised and guarded against.

In a medical setting, the cost for failure in a models returned answering is asymmetric. Wrong answers to general knowledge medical questions could be a inconvenience, whereas an incorrect dosage amount, inverted diagnostic criterion is not at, and is a very serious error that could lead to catastrophic effects, thus not all questions are treated the same in importance in practise. Although with the benchmarks, a multi-choice questionnaire accuracy score does not really distinguish between importance of questions, for example a query on what is cancer and what is the correct dosage of ibuprofen may be measured the same. Research has begun into the construction of benchmarks that monitor and measure medical hallucinations and show that relationship between a models factual grounding and fluency behaviour are only loosely coupled. Pal et al. (2023) Thus, a model cannot treat all medical questions the same with the same risk and importance weighting.

Furthermore, it has been discovered that medical models are poorly calibrated with respect to their own limits and internal capabilities. When investigating whether or not a medical model can identify the extent and boundary of it's own knowledge has yielded inconsistent and struggles to determine whether or not a medical question can reliably be safely declined, which is a problem in a clinical domain as the capacity to decline a question is a safety procedure and not a deficiency of the clinical reference. Kadavath et al. (2022)

These three additional insights reframe the clinical questioning and answering design problem. It is not the objective that the clinical model holds more domain medical knowledge, since none of these insights are problems of quantity of training data. It is however a system that defines what can actually be retrieved and supported from a reliable and identifiable medical source, and one that guards and declines questioning where no current reliable source exists. Rather than the quantity and quality of a models weights, it can be said the surrounding architecture that finds if a question has a accurate and safely grounded answer is much more important when dealing with medical questioning.

2.2 Small language models and their limits of parametric factual clinical recall

The models that are discussion in Figure 1.1 from the previous section, operate on hardware at a scale that precludes local on-device edge execution. Thus, a parallel line of development has been drawn that deliberately targets compact models, usually referred to as small language models that are designed and intended to both run on consumer and embedded edge IoT hardware and devices. While the boundary for these small language models is not universally nor formally agreed upon, they usually range from models from below one billion parameters to below eight billion parameters. For the purposes of this report, the class of small language models is defined by the problem of deployment constraints rather by conventions of the small language classification. Thus, small language models, that fit, after applying quantisation, within the working memory footprint of a 2 GB edge IoT device alongside their associated generated retrieval index and runtime binary. Using quantisation at 4 bits on 1 billion parameter takes up approximately a memory footprint of 770 MB, which leaves a comfortable and adequate margin on a 2 GB device, whereas a larger 3 billion parameter model does not and can not.

Small language models are not a niche or marginal category of language model. Major language

model families such as Microsoft's Phi-3 model Abdin et al. (2024), Google's Gemma family of small language model Team et al. (2024), and the 1 and 3 billion parameter open weight variants from Meta's Llama 3.2 family now all target and cater for the deployment of models on sub-billion parameter edge targets such as the 2 GB Pi 5 baseline. Grattafiori et al. (2024) These models are designed and built to work on the memory constrained devices and thus clearly show the language model field clearly takes such on device edge targets seriously.

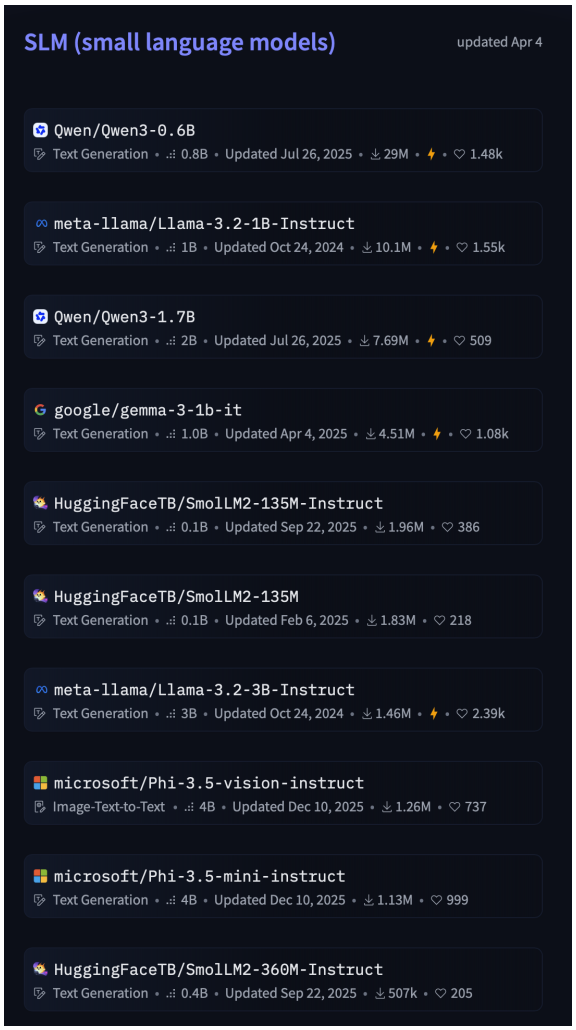


Figure 2.1: A collection of the most downloaded small language models on hugging face, note the downloads in the , showing clear interest and demand for SLM's Ji (2026)

What such small language models cannot do compared to the larger parameter large language models is hold and recall large bodies of specific facts. As the specific knowledge all language models can recall on it's own is bounded and constraint by the parameters available to the model to store it, which is a relationship established by directly measuring how much factual content language models of increasingly sized parameters can be made to reproduce and store. Raffel et al. (2020) This specific scaling relationship is consistent with the broader domain relationship scaling laws between a models parameters, target corpus data to recall and loss. Kaplan et al. (2020) Hoffmann et al. (2022) Thus a clinical corpus, covering several thousand distinct diseases, symptoms, drugs and conditions, each paired with multiple appended aspects, greatly exceeds

what a raw small language model of this edge size can encode with any reliability.

Continuing to train a model on more training data brings with it diminishing returns once that particular models parameter budget is the binding constraint on knowledge intensive retrieval tasks.. Thereby, counting to train a small language model on more and more domain medical prose improves it's fluency and structure of the returned answer although the actual factual contents of said answers may not have been necessarily improved. This particular relationship is easy to miss as it states a model that has become more fluent on a given input medical corpus can read and behave as though it has become more knowledgeable on the domain although returned answering may not have actually factually improved.

The notion that a models capabilities do not degrade uniformly in scale in regards to a models parameters is a contested area of research. Particular model capabilities have been reported to appear only above particular model sizes Wei et al. (2023) although continued further research argues that under sustained and continuous scoring these metric artefacts largely disappear. Schaeffer et al. (2023) This shows that while reduction in a models capabilities purely uniformly is not a dispute, certain model capabilities survive model compression much better such as the grammaticality, tone and formality and output format adherence compared to the recall of specific model facts, which degrades much more strongly during compression. Therefore, the implication for a small language model design must be to remain entirely adequate as a component and structural layer responsible for the presentation of clinical and medical answers while being much less inadequate as the actual ground source of the retrieved answers factual contents.

This observation is core to where the EXCALIBUR-Medic's architecture rests. As if the main constraint and limitation in a small language model is specifically the type and scope of data of what it holds, rather than what it can actually infer from the material retrieved and placed in before it, thus the correct approach and response is not to force train the knowledge it lost during compression back in, but it is to supply the specific factual knowledge from elsewhere, when such knowledge is needed, and to use the compressed model for what compression leaves intact, such as the aforementioned knowledge of knowing the tone, formality and structural compliance.

2.3 Model compression techniques; distillation, fine-tuning and quantisation

To produce such a clinical model that fits into the constraints of a small language model under 2 GB footprint as defined in the previous section, whilst also retaining useful model behaviour i.e. formatting, the model must follow an distinct, 3 stage compression pipeline in sequence to ensure the best compression result.

The first stage, known as knowledge distillation trains a smaller, compact student small language model to try and reproduce the behaviour of a much larger teacher large language model rather than fit the original clinical training labels directly. Hinton et al. (2015) The original incarnation of this technique, pioneered by Hinton et al in 2015 for classification models was quickly expanded and transferred readily to encoder language models with DistilBERT in 2019. Sanh et al. (2020) When applied to generative models it differs as what is transferred through distillation is not the

probability distribution over a target label as what would be found during when applied to classification, instead what is transferred is sequences of text, with effective distillation formulations bringing over and transferring down from the teacher to the student, the teachers intermediate reasoning steps on top of the retrieved teachers answer to said prompt. Building on top of this, training the student model using teacher-generated chain-of-thought has been shown to improve small language model distillation performance beyond just targetting distillation on the teachers answers alone achieves. Ho et al. (2023) Magister et al. (2023) Hsieh et al. (2023) In practise this means the knowledge distillation performs as follows; using the larger medical teacher model in effect as a generator of training data that would otherwise require human annotation, thus the reasoning and answering pairs from the teacher gets distilled down and trained on the smaller student model.

This chain-of-thought distillation approach has different variants in what the student model is trained against, for example sequence-level approaches fit the student model onto the reasoning text output that the medical teacher model has produced which makes this approach affordable, as the teacher only needs to run once offline, to generate the output and then it's output can be reused on many student models. Whereas, a on-policy approach instead uses samples from the student model during training and scores the students samples against and the larger medical teachers distribution, removing the mismatch between the sequences a student is exposed to and trained on from the teacher and those of which it will actually generate at runtime. Lobo et al. (2025) Finally, a third variant of this approach exists, preference-based methods form the third family of chain of thought distillation, where the student model is optimised by ranking pairs of generated outputs rather than against a reference model. Rafailov et al. (2024)

The time to train even a 1 billion parameter model from scratch is compute-costly and time-consuming even on high-end consumer software. Therefore the use of parameter-efficient fine-tuning can be used by freezing the pre-trained weights of the student model and by appending and training on a small number of additional parameters onto the student model. This avoids having to train every single weight in the model, and instead low rank adaption can decompose the weight updates of the student model into a pair of low-rank matrices carry the adaption changes and the rest is untouched and thus the entire model does not need to be retrained. Hu et al. (2021) Thus efficient fine-tuning can be done on consumer hardware and the trained adaptions from the fine-tuning process can be merged down into the vanilla base weights of the model, allowing for the creation of one single fine-tuned model with no extra inference runtime cost or overhead. Houlsby et al. (2019)

Reducing the precision of which the weights are stored within a given model is the technique of Quantisation and is the method that allows a one billion parameter model to be run within the memory that is available within the 2 GB Pi 5 target device, without quantisation the model would be too big to fit into memory on the target device. Quantisation occurs after the student model has been trained, and a model can easily be quantised without having to retrain it from scratch. There are several different ways quantisation can be performed, outlier activations layers can be found and isolated Dettmers et al. (2022), or alternatively the quantisation process can preserve the activations that have been identified as mattering most and removing outwards. Nobari et al.

(2025) However what is important to understand is that the relationship between the bit width of the quantisation, i.e. to how many bits the model is quantised and the corresponding quality of the model is not linear, and in fact fewer quantisation with fewer bit widths perform more strongly than intuition would suggest, even at 4 bit quantisation. Dettmers and Zettlemoyer (2023)

Table 2.1: The three compression operations applied in this work.

Technique	What it does	What it does not do
Knowledge distillation	Transfers the behaviour of a larger teacher into a smaller student, including its intermediate reasoning.	Add facts the student has no capacity to store.
Parameter-efficient fine-tuning	Trains a small low-rank adaptation while the pre-trained weights stay frozen, making training affordable on consumer hardware.	Alter the capacity of the model being adapted.
Quantisation	Reduces the numerical precision at which trained weights are stored, bringing the model within the device's memory.	Change what the model knows.

What is important to understand about these compression steps and techniques is that none of them actually add any knowledge to the model, wherein distillation primarily transfers the behaviour of the teacher model onto the student model, fine-tune parameter-efficient lora allows for affordable training onto the student model rather than a full-retrain and quantisation most importantly reduces the storage cost of storing the resultant student model. The main finding of the previous section, that being small language models don't have the parameter space for storing and holding the factual content a clinical corpus requires and contains is thus untouched by these compression techniques, as such a solution is addressed in the next section, that being retrieval.

2.4 Grounding medical query answering generation in retrieved clinical corpus text

If the main deficit of a small language model is what it can hold rather than what it can do with the source material, then the response is to retrieve and place the material before it. RAG which is short for Retrieval augmented generation positions and conditions the model on documents fetched at inference time, so the content of generated answers from a users queries originates and is sourced from a identifiable source in the corpus rather than the probabilistic weights of the model. Lewis et al. (2021) As RAG can be implemented into the inference time of the interactive chat student model it does not require to be trained or re-trained with the retrieval logic, thus retrieval requires no internal changes to the model, additionally allows the clinical corpus to be updated without retraining on it and it makes the model generated answer source available for inspection as it was retrieved from somewhere rather than the models weights just making the answer up.

This type of retrieval can be divided in two distinct families, retrievers that use a dense approach firstly encode the queries and documents all into one shared vector space using a trained encoder and retrieve from nearest neighbour search inside the vector space. Karpukhin et al. (2020) On the other hand, sparse based retrievers work by scoring document through which terms overlap, the most well known of sparse retriever is the BM25 ranker, which ranks documents by summing

up the per-term occurrences and contributions and combine a inverse document frequency rate of the terms with an additional component that saturates term-frequency so that a diminishing returns penalty is applied to repeated procurances plus length normalisation that prevents extremely long documents from scoring merely by verbosity and length alone. Robertson and Zaragoza (2009)

As clinical questioning requires a retriever that can handle many heterogenous tasks, such as general medical questioning, specific drug dosage questioning among other specific disease symptom analysis questioning, and as such additionally requires a retriever to function at inference time i.e. no specific domain pre-training, it makes sense to select a sparse retriever such as the aforementioned BM25 retriever as the BM25 provides a inspectable scoring of each contributed term which is helpful in matching user vocabulary to conditions and drug names which are often compound and additionally the BM25 retriever requires zero training and thus the behaviour of the retriever cannot change nor drift unlike the dense retrievers. Thakur et al. (2021)

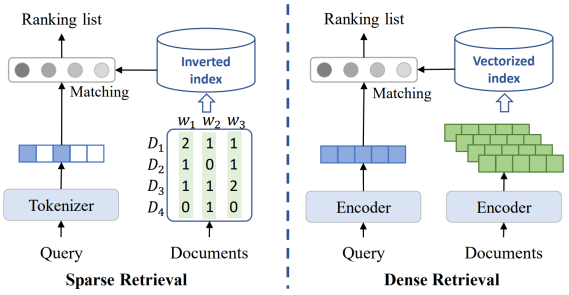


Figure 2.2: Sparse vs dense retrieval Sumit (2023)

Additionally, the deployment edge device constraints reinforce the same retriever conclusion. Dense retrievers require an extra embedding model resident alongside the model itself, on a edge device where memory is already so mission critical the additional cost for the extra embedder is not affordable. As the BM25 only requires a simple standard index, and thus the evidence and constraint driven choice coincide and both point to sparse retrieval being the better choice in this particular scenario.

The source material to be retrieved by the BM25 retriever and what will be compiled in the supplementary retrieval index for EXCALIBUR-Medic must be authoritative, clinically accurate, come from a legitimate source and there are three main major public sources that can supply it. The MedQuAD dataset provides general clinical question and answering pairs collected from the US National Institutes of Health websites, where the dataset is organised by conditions and by question type. Abacha and Demner-Fushman (2019) The open-FDA programmes supplies the data of drug labelling in it's structured form, carrying dosing information, drug warnings and drug interaction sections that most clinical questions most often concern. Lastly, the MedlinePlus corpus provides general patient consumer-oriented health topics, more general topics that the ones the specialist MedQuAD and open-FDA do not cover. Miller et al. (2000) All 3 corpses are either public domain or freely accessible and redistributable, ensuring the device can legally hold them on device locally.

Table 2.2: Dense and sparse retrieval compared against the criteria that matter for on-device clinical retrieval.

	Dense retrieval	**Sparse retrieval (BM25)**
Matching	Semantic similarity in a learned vector space.	Lexical term overlap, weighted by how discriminating each term is.
Training	Requires a trained encoder; behaviour depends on what it was trained on.	None. Deterministic, and cannot drift.
Cost at inference	An embedding model resident in memory alongside the generative model, plus a vector index.	An inverted index and arithmetic available in any standard library.
Out of domain	Frequently fails to outperform BM25 without in-domain training.	A robust zero-shot baseline across heterogeneous tasks.
Inspectability	The score is a distance; the contribution of individual terms is not recoverable.	Each term's contribution to the score can be recovered and examined.

However, retrieval by itself does not solve the problem fully, as the retriever will always return a ranked list no matter if the queried input is clinically relevant or not. There exists no current query for which the retriever returns nothing and as we want the clinical model to only answer medically relevant questions this is a issue. A failure therefore of the model and the retriever as it currently cannot determine what it does not know as what was previously explained with the clinical benchmarks insights. Therefore, the resultant EXCALIBUR-Medic model will need to introduce guards that restrict only the outputs that the model knows for sure, rejects non-clinical queries all before model generation output actually begins.

2.5 Safeguards for interactive generative clinical questioning and answering systems

To prevent a model from answering non-medically relevant queries and or medical clinical facts that the model does not fully feel is answerable, the following safeguard mechanisms are available and operate within the models generation process during prompting to ensure safety and reliable accuracy in the domain of clinical questioning and answering from a users given prompt query.

Rather than answer unreliably a users prompt, the model can instead perform abstention, that is the practice of declining to answer that specific users query and tell the user why the query was refused rather than give out the wrong answer or answer a medically irrelevant query. This concept first appeared long before the introduction of language models where in the domain of classification tasks, abstention acting as the reject option for when a classification goes below a classifiers confidence threshold. Chow (1970) Instead of applying abstention to the model generation where all the training data would need to be precomputed with thresholds and where the model itself has trouble accessing whether or not it actually knows the answer, abstention and thresholding should instead be applied to the retrieval instead of the generation. Yin et al. (2023) As RAG already measures how well a retrieval matches the query, how much of the query is actually in the source, a thresholding value can be computed and measured against, meaning retrieval now handles abstention and can determine whether or not to actually answer the question or decline it depending

on how confident it believes the threshold of both the user query and the returned retrieval to be.

A further safeguard would be the technique of constrained decoding, that is the practise of restricting the tokens the model may emit so that malformed model generated output becomes impossible rather than an unlikely possibility. Every output the model generates must conform to a defined model output structure, where only the permitted continuations at each step of the generation can be drawn and computed from a defined formal grammar such that the remaining probability is mass renormalised over the formal structure, ensuring and guaranteeing the model can only produce the well-formed and defined structured output by construction. Geng et al. (2023)

The third and final mechanism procedure is of attribution, where the exact relationship between a generated model answer and it's source truth is explicit in nature and verifiably checkable. This means that every generation by the model from a query is attributed, that each property of a generated statement should be accompanied by and supported by an identifiable and traceable source from the indexed corpus and proposes means of measuring whether such a identified source and whether it holds, i.e. it actually can be identified and retrieved from the ground corpus. Rashkin et al. (2023) A limiting case of the attribution safeguard is concerning verbatim retrieval quotation, that being, if a user asks for a safety-critical dosage query, the returned output is reproduced without paraphrasing anything, giving the exact mission-critical information the user requires rather than incorporate the models own paraphrasing that could misinterpret the verbatim definition, although losing the fluency and length checks that the model would otherwise strictly enforce. This trade-off is worth it as verbatim mission-critical queries are precisely answered without biases or malformity that could arise with a model generated answer and ensures such safety-critical questions can only be answered whether or not the correctly passage in the corpus can be found or not, if it is not the model does not make up an answer, it tells the user straightly that no valid verbatim answer was found, completely bypassing a scenario where the model could generate hallucinations in the most safety-critical of user queries.

Table 2.3: Three safeguards, and the point in the process at which each acts.

Safeguard	**Acts**	**Governs**
Abstention	Before generation	Whether the model answers at all.
Constrained decoding	During generation	The form a answer could take.
Attribution	On the answer	Whether a answer is traceable to its grounded source in the corpus.

These three countermeasures compliment both the core distilled model behaviour and runtime answer retrieval augmentations. Implementing abstention ensures and governs whether or not the system actually gives an answer to a query, constrained decoding ensures and governs the form and structure of the answer is well-formed and compatible with the model and lastly attribution ensures all given model generated answers can be easily identified with it's source corpus text and the clinical answer was not just made up by the probabilistic weights of the model.

2.6 Background summary

This background chapter has defined the constraints of the aim within the domain, the exact roadblocks and bottlenecks that such a on device edge interactive clinic model faces, along with a background of the architecture EXCALIBUR-Medic ultimately adopted in its design and implementation.

A closer examination of models clinical benchmarking performance showed that rather than unaided generation, the models were exposed and supplied to multiple-choice questioning and do not fully explore the failures that matter most in a clinical domain. Furthermore, models can and do often generate confident answers of which they don't actually understand if they are in the boundary of what the model can and cannot answer reliably. This informed that the objective is not purely a model that knows much more clinical knowledge, but a smarter system that only answers queries of which it can support the reasoning and has the source to.

The background on small language models also showed that to fit a model within a 2 GB memory footprint, it cannot hold the quantity of domain facts that a clinical corpus expects and contains within it's limited 1 billion parameters and compression techniques do not alter that constraint inherent limitation. Using knowledge distillation transfers the more robust model behaviour from the teacher model, lora methods allow for transfer learning without having to retrain every layer exhaustively and quantisation allows a model to fit onto the limited memory pool of the target device. This compression pipeline offers a competent model handling of retrieved material placed before it and is not the same capability of actually holding the corpus facts.

From the compressed model inability to hold the sufficient amount for parametrised clinical facts, allows for retrieval to supply those facts instead. Using a sparse BM25 retriever ensures that retrieval does not increase the limited deployment target and ensures retrieved answers are both inspectable and traceable and free of drift as no domain training has been applied to the retriever. And as the any query would prompt a retrieval, the system implements abstention to determine whether or not a query can actually be answered or not through confidence thresholding of the index.

Table 2.4: The chain of argument developed in this chapter, and what each step rules out.

	Establishes	**Consequence**
§2.1	Benchmarks measure discrimination among supplied candidates; failures are fluent and carry no surface signal.	The objective is a property of the system, not a larger model.
§2.2	A model within 2 GB cannot hold the factual content of a clinical corpus.	Knowledge must come from outside the weights.
§2.3	Distillation, efficient fine-tuning and quantisation transfer behaviour and reduce cost.	Better compression is not an escape route.
§2.4	Retrieval supplies the missing content, but returns a ranked list for every input.	The decision to answer cannot be delegated to either component.
§2.5	Abstention, constrained decoding and attribution act at different points in generation.	The system governs whether, how, and from what source it answers.

Chapter 3

Literature Survey

3.1 Medical focused large language models

Domain-adapted medical/clinical large language models can be produced and created through two broad and distinct routes. The first route, is taking a general language model and pursuing continued pre-training on medical text and literature. AN example of such a model trained via continued pre-training is a model known as MEDITRON which extends a general language model and enhances and extends it's medical reasoning and factual knowledge by additionally training on a curated clinical corpus such as medical guidelines and institutional biomedical literature. Chen et al. (2023b) Two additional models named BioMistral and PMC-LLaMA are two open weight models that use the same principle of continued pre-training on medical literature but in this case these two models use the specific PubMed Central corpus for their improved medical knowledge and reasoning, thereby extending this practice to open source models. Labrak et al. (2024) Wu et al. (2023) The other practise of generating medical language models is by applying instruction tuning based on a supplied question-answer medical pair to the model during training. One of the early models to perform medical instruction tuning was MedAlpaca leveraging over 160,000 such medical question-answer pairs. Han et al. (2025) To ensure such fed medical questioning actually have a verifiable medical answer, the model, HuatuoGPT-o1, creates a verified for said instruction tuning medical pairs to ensure and guide the research for verifiable reasoning trajectories thus insuring the corresponding training from instruction tuning is even more reliable in it's medical reasoning. Chen et al. (2024) Turning to proprietary closed weight models, Med-PaLM also invokes and uses instruction prompting on a extremely large base model of 540 billion parameters, showing the procedure clearly scales. Singhal et al. (2023)

A clear problem identified with this literature was the problem of model scale, where the smallest of such recorded models was a seven billion parameter model which is approximately the smallest such scale at which competitive reasoning on clinical benchmarks has currently been demonstrated and reported. Publications show that these open weight models usually start from the 7 billion parameter range and can scale to 70 billion and above.Sim and Chen (2025) This is a clear problem as the target model developed in this work is a one billion parameter model and is thus seven times compressed and smaller than the smallest model reported in the aforementioned reports, which places the target model parameter size outside the range of all these recorded results, thus specific benchmarking and evaluation of a one billion parameters model medical performance current does

not seem to exist.

Another finding from the surveyed literature highlights the problem is that these practices intended to adapt a model to medical performance often account for less than the marketed performance claim. As mentioned previously, many general purpose models often and frequently match or even exceed what a adapted medical-specific model at comparable parameter sizes. And even literature exists that brings skepticism to some of the claims of these medically adapted models. Jeong et al. (2025) Thus, when a model is trained on clinical corpus's that contribute less to the medical performance than pure scale does, then medical adaption strategies applying to even smaller and sub seven million medical models is unlikely to reach the constraint of fundamental and arbitrary parametrised knowledge gaps required for a medical domain.

Instead of the work directly adapting a small language model on medical text, the work diverts from this surveyed approach and instead uses a medical large language teacher model and transfers via distillation it's medical behaviour into a much smaller one billion parameter student model and as mentioned before what is transferred slash adapted is not medical knowledge fully itself but the models behaviour of said medical data.

3.2 Retrieval augmented powered clinical questioning and answering

To answer if retrieval can actually improve medical questioning and answering performance, the work shows that such a procedure is established in literature rather than contested conjecture. The first justification of such an retrieval approach in practise, is the benchmark of MIRAGE, of which contains 7,663 compiled questions drawn from a source of 5 medical questioning and answering corpus datasets. Xiong et al. (2024) The results from the MIRAGE benchmark shows a clear win for retrieval based approaches, with the retrievers averaging a yielded improvement rate of between 1 and 18 percent over the rival chain of thought prompting approach using the same exact models.

Retrieval can also greatly enhance a general model, a clear example being of when a GPT 3.5 model was augmented with RAG, it displayed performance comparable to a GPT 4 without retrieval in the domain of medical performance. Xiong (2024) This means that retrieval can act as a substitute for model scale and reach a capability that would require a larger model but instead can be supplied with a smaller less capable model when supplied with relevant retrieval. Which is exactly the premise of which the work pursues and while the premise is not original to the work, it is still untested on how far down on model sizing the benefits of retrieval can still extend to.

From this literature, the choice of what constitutes and what are the sources of a retrievals domain of corpus was found to be critical. Instead of just one source, a corpus must be compromised of many sources of different tasks, i.e. general medical knowledge plus hyper-specific diseases and drug catalogues plus examination style questioning help ensure the retrievals can be robust as if only one corpus is supplied as a source, the retrievals would be domain and task bounded, whereas a compiled corpus compromised of many sources can answer more robustly and is less bounded by one particular or specific niche medical domain.

One problem with the findings of these retrievers is that memory was not a focused constraining factor, therefore there is still a gap to determine whether or not a edge AI deployment where the 2 GB memory ceiling is the first and foremost constraint, as many of these retrievers were hosted on server scale hardware where memory was plentiful and not a studied benchmark. So it is still in contention whether or not retrieval substitution to a models inherent capacity will still hold at the extreme of memory constraints. As the memory ceiling of above retrieval substitutions was beyond the memory ceiling of what the works one billion parameter model is constraint upon and this shows potential testable claims that the work can make at this scale of memory ceiling.

3.3 On-device small language model edge deployment

As previously mentioned, there is an entire ecosystem of small language models marketed by their capability of running on even the smallest of edge devices, such as smartphones and single board computers, such as TinyLlama, which was trained specifically as a open weighted one billion parameters inspired by the architecture and tokeniser of the Llama 2 family of large language models. Zhang et al. (2024) There even exists small language models that are sized below one billion parameters, in a few hundred of millions such as the SmolLM family of small language models which ranges all the way down to a staggeringly low 135 million parameters. Allal et al. (2025) And even the popular Qwen 2.5 family of models has released a model that has half a billion parameters. Qwen et al. (2025) Some small language models also market themself as being competitive with substantially larger large language models from itself, such as the MiniCPM model variants of 1.2B and 2.4B which they claim are competitive with 7B to 13B large language models. Hu et al. (2024) All of these impressive and small language models are underpinned with the need for a ultra optimised runtime, usually the c++ written llama.cpp which leverages hardware optimisations that would not be possible on without it such as SIMD acceleration and built-in quantisation that would otherwise not be available when using higher-level runtimes. ggml org (2026)

While these surveyed literature handle and reach the same parameter range as what the work requires, their principle evaluation concerns general runtime and memory performance with less of an emphasis on domain relatability, that being clinical in this case and sustained and extensive energy monitoring and measuring. As these small language models perform usually as general assistants and there is little literature when they are instead domain specific assistants, thus there is a clear gap in how abstention of domain critical answering and how a model defines such a threshold in the findings.

Furthermore, while the findings do account for rigorous quantisation, for example a 3 billion parameter when exposed to 4 bit quantisation compresses down to 1.8 gigabytes of weights, and for a 2 billion parameter model hovering around 1.2 gigabytes, these findings as they are solely focused on the model half and not any retrieval augmentation do not account for the additional retrieval index cost on memory and storage space. Furthermore, how much a operating system such as GNU Linux also consumes in addition to the model deployment is often also not reported. Thus factoring in the Linux and additional runtime costs of key value caching and retrieval artefacts means that the practical memory ceiling of the work is closer to the one billion parameter sized models rather than the frequently quoted two or three billion models when factoring in real world

costs of operating system et al.

3.4 The gap identified from the surveyed literature

Although the surveyed medical large language models establish clinical capability, the smallest of these measured models is seven times larger in size than the deployment target and memory ceiling that this work permits. Additionally, using retrieval reveals that such retrievers can substitute for model parameter scale, although such findings show that the substitution occurred in such a range in which memory was never the number one reported constraint. Furthermore, on device deployment strategies reveal that small language models can run on constrained hardware, but often benchmarking from these models is focused purely on token throughput rather than specific domain answering reliability, which is extremely important in the context of the clinical domain.

Table 3.1: Key surveyed literature findings against the strict target constraints of the Pi 5 2 GB deployment target edge device.

Area	**Model scale**	**Retrieval**	**Within 2 GB**
Medical LLMs	7B–70B	None	No
Medical retrieval augmentation	Server hosted	Dense + encoder model	No
On-device SLM edge deployment	0.5B to 3B	None	Marginal
EXCALIBUR-Medic Model	1B	Sparse; no encoder	Yes

All three findings from the literature stop short at the strict constraints of which this work intends to deploy to. While retrieval substitution may have aided a GPT 3.5 model to perform like a GPT 4 model, such literature does not exist to currently show if such substitution will also exist when applied to a one billion parameter model and also does test against and tested memory ceiling deployment constraint, nor the exact distribution of model capability of which is the model itself and which is retrieval at such a sub 2 GB memory ceiling. And these are the exact gaps and questions the work intends to answer.

Chapter 4

Design

4.1 The EXCALIBUR-Medic model pipeline overview

The EXCALIBUR-Medic model will be designed in one of four ways to answer a users given clinical prompt. First it can abstain from answers and decline if it believes it does not have the confidence nor source to answer a clinical question or refuse a non-clinical domain question altogether, it can then offer and return a set of more specific clinical questions to aid and better specify more generic of user queries, it could also quote verbatim source passages when dealing with safety-critical queries such as dosage amounts or it could more generally generate reasoning and answering from a retrieved reference in its known corpus. These model decisions, of which one is selected is determined before any text generation occurs.

Such a design paradigm was selected and determined from the importance of safeguards, gathered in 2.3. When models could not sufficiently determine whether or not they actually believe to the best of their abilities to answer the clinical question reliably and factually, and blindly answering a clinical question without a confidence index nor abstention capabilities can and will present unreliable clinical answers that can harm the user. Thus, retrieval governs and safeguards before a single word of text is generated by the model proper. Including how strongly the user query subject matter corresponds to the EXCALIBUR-Medic's corpus via document matching scores, determining how much of the user question the ground truth medical corpus accounts for and whether the user asked about subject matter even exists in the known corpus at all. These safeguarding computations are once again made before text generation begins, meaning that every decision the model makes, text generation wise, i.e. whether it can reason and answer the input user clinical query rests on these safeguards firstly and foremost.

The resultant pipeline requires two main components to be designed. Firstly, the EXCALIBUR-Medic model itself. Which is a smaller one billion student model distilled from a much larger medically fine-tuned teacher model into one compressed enough to both deploy onto the target 2 GB memory footprint and one that is trained, learnt and distilled from the teacher model, that can exhibit the behaviour and structure of such a reasoning and answering the rest of the pipeline depends on and expects. The other component, the model's retriever and it's required sub-systems, the compiled medical corpus, the retrievers ranking function and it's corresponding decision layer and verbatim answering routing is also designed, and requires all sub-components to both share

and work in tandem for the entire retrieval process to function and return reliable medical reasoning and answering.

The below figure 4.1 shows an abstract view of the aforementioned two components, retriever and model, and their child sub-components. From this figure, a user query prompt is first taken as input as the question, normalised to extract any specific brand names or other key information such as a particular disease or symptom and maps any known aliases, normalise troublesome grammar such as apostrophes and checks whether a question type can be determined. This normalisation process returns the normalised question and passes it to the retriever, a BM25 that that scores the passed input normalised query against an inverted index with it's specific domain clinical modifications criteria for better matching conditions and clinical type of questioning in the source clinical corpus. The retriever then returns the single best matching clinical document returned from said modified criteria. The best-scoring retrieved document found by the retriever and it's scoring results is then passed to a final decision layer comprised of five specific guards, of which the five guards can divert the user query question away from generation if it does not deem it questionable. A user clinical question query is thus only passed to the model and is answered if it survives and passes all five safeguards.

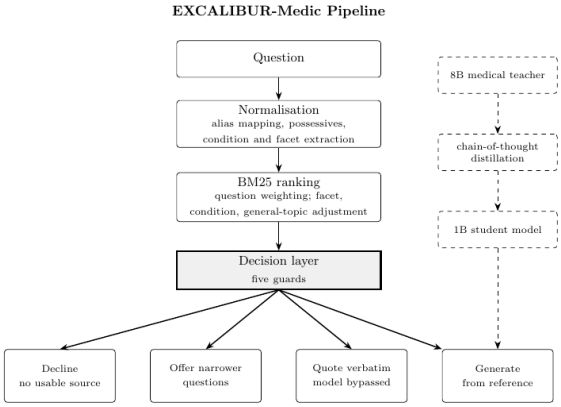


Figure 4.1: EXCALIBUR-Medic Pipeline, showing the interaction of both retrieval and distillation of model, on how a question answering is actually reached and pipelined, note the shared responsibility for answering.

A design quirk of such a pipeline is that the model itself is only actually reached as the last component and not the first as the retrieval process both handles and safeguards the user query at hand firstly. Thus the EXCALIBUR-Medic model itself can only actually receive and view a user question if it the preceding retrieval component has deemed it both valid, clinically relevant and safe to both reason and answer with. Additionally, no component of the pipeline itself is solely responsible for the decision of answering. As the pipelines retriever determines and deems the input users question scoring, the safeguards check and applying if the question is therefore domain relevant and if in fact a reliable and truthful question can be produced, and the model itself only writes the generated text from the safeguarded text it has been given and as such require all components and subcomponents to interface and interact in order to serve and answer the user, rather than a binary yes or no stage.

4.2 Training the distilled excalibur model

As stated before when discussing the compression methods in the background section, the main intention of the distillation process is not to produce a distilled student model that knows more medicine facts. Instead, this distillation process is intended to and designed for providing the student model with a base and better set of behaviours inherited from the teacher model, outputting reliably and faithfully from source material placed in front of the user query, structuring the model in a way the rest of the pipeline can validate and successfully parse, with reasoning and answering behaviour blocks inherited from the distilled from teacher and declining when the model has been instructed that no source clinical ground truths currently exists.

This focus on distilling model behaviour and structure rather than traditional source knowledge shapes and determines how the teacher model is chosen and selected, a medical model that output well presented answers but drifts wildly form the source clinical text is the wrong training signal to test for. Instead candidate teacher models are judged on both fidelity and how traceable i.e. reliable the generate output tokens are to the referenced source text it was visible to. Medical models that are more reasoning focused are therefore the wrong kind of model to selected as a candidate as despite these models generative long and expressive summaries of a given clinical prompt, they produce outputs widely different and often unreliable, hallucination prone that in this clinical domain is problematic to deviate so far from the truthful source prose. Instead, this pipeline requires a candidate teacher model that acts more like a stenograph, not a thinker. As the medical facts are already grounded in the reference medical text, the model therefore only needs to copy the reference text and not change any medical facts or knowledge. The target corpus that will be used by the medical teacher to generate said chain of thought reasoning and answering pairs is the MedQuAD dataset which will be the target corpus of which all training will be applied to and fed to the teacher model to generate the grounded pairs from.

The knowledge that will be generated and distilled down to the student, is create through supervised fine tuning, of which the teacher will produce reasoning passages and answering passes via chain of thought generations on a massive scale, passing in clinical source query and answering pairs from the reference clinical corpus to ground these such generations in truths. These pairs will be identifiable and parsed via tagging in the teacher models output. This carries on from the background finding that the teacher models intermediate reasoning and answering steps are what actually improves what the smaller student clinical model learns. Secondly these two reasoning and answering tags allows for styling in the text prompt to better differentiate the generated output of which the models reasoning and actual answering of a users clinical query, furthermore an atomic answer tag corresponds directly to retrieval answers in the source code, allowing for a more logical and structured mapping from ground truth answer to the end generated answer token block.

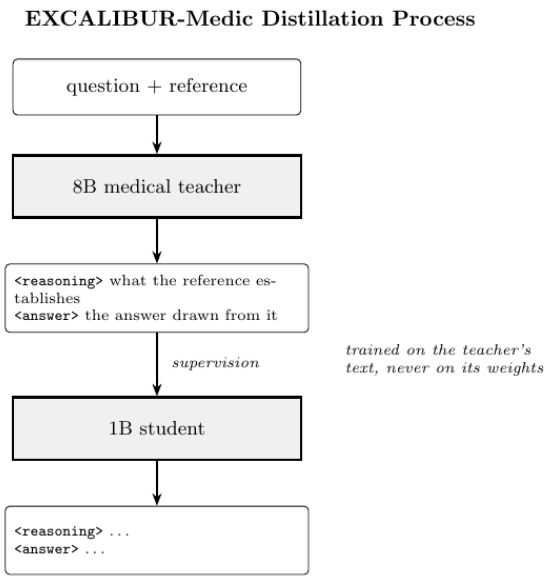


Figure 4.2: An abstract view of the distillation process before the quality gate, note the reasoning and answering pairs behaviour distilled down from the medical teacher model onto the student model.

Raw output from the teacher is not used as directly generated, as a quality gate determines and rejects teacher generated samples on the follow criteria identified in the below figure 4.3. The first judgement is length, this ensures generation pairs that are distilled to the student is bounded and stripped of any verbose or too short answers, of which can exist in the source corpus as answers made up of many tabular references of which the end user does not want to see as a natural language output answer to their question. The secound restriction and judge of the quality gate is refusal language, where the teacher declines a question it should also distil down to the student to also tell the student to decline and refuse such a question. Repetition collapse and reference overlap ensure the generated teacher outputs are quality gated to actually check if the generated answer and reasoning pairs are reliably identifiable and traceable in the source clinical text. This quality gate is very strict on purpose, as defections in the distilled corpus must not exist as unreliable distilled generations cannot be corrected or fixed after distillation, therefore the quality must be enforced here before the student even gets a drop of distillation.

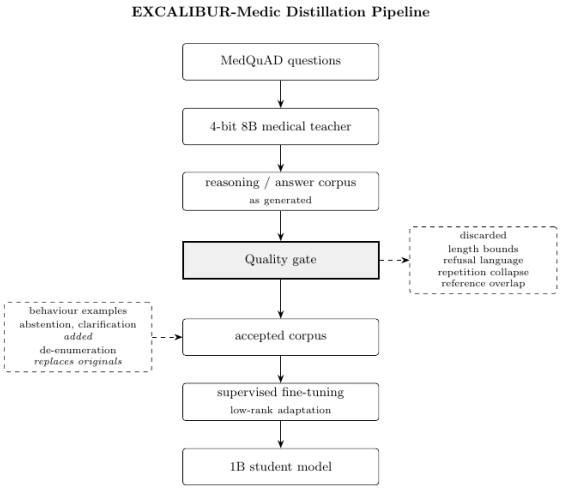


Figure 4.3: The distillation pipeline of EXCALIBUR-Medic

Furthermore, an additional behavioural training component was built on-top to handle and enforce strict behaviours that the medical teacher alone cannot supply. The first sub-component, abstention enforces whether or not a model can decline if no reference is found in the source clinical corpus rather than revert to answering from its in memory, potentially unreliable and inaccurate medical parametrised knowledge. The next behavioural sub-component, and judge is clarification, which checks if a overly broad user question is supplied and supplies instead a request to narrow down what the user was actually enquiring on, such as if the question asks broadly how to treat cancer, the model will return, what particular cancer did you mean, skin, lung et cetera. The final additional behavioural sub-component de-enumeration, which ensures that source answers that contain lists of many itemised elements i.e. references to clinical sources do not get treated as one single answer. Within the training mixture, the first two behavioural sub-components only take a small share, as a model that is mostly comprised of guards that refuse generates will likely arise behaviour that refuses answering when it really should not. On the other hand de-enumeration placates and replaces these many itemised source clinical answers to ensure the training behaviour is settle on these shorter, de-enumerated list source texts rather than noisily training on both.

Additionally, it is worth stating that the additional behavioural training is derived upon the acceptable output as judged from the quality gate rather than reg-generate teacher generation again which would cost double the computation and this approach of applying behaviour from the quality gated output allows behavioural training that is affordable as instead of retraining the teacher itself, the behaviour only requires regeneration from the teacher medical model. The distilled smaller student model is adapted rather than fully retrained from scratch, lora targets a subset of the students layers rather than all of the student layers and this allows for the distillation to be done in a reasonable amount of time.

4.3 Constructing and assembling the clinical retrieval corpus

To ensure the EXCALIBUR-Medic model will be able to answer a broad range of clinical questioning, the retrieval corpus will be assembled, compiled and composed of three open source and

legitimate medical corpus sources. All together they will contain well over 19,000 clinical, reliable documents. The 3 sources compiled for the retrieval corpus are defined in the table below, 4.1.

Table 4.1: The three corpus sources.

Source	Covers	Contributes
NIH QA pairs	Named clinical conditions, organised by condition and type of question.	The majority of documents; several thousand specific medical conditions.
openFDA drug labels	Dosing amounts, safety warnings, drug interactions and pregnancy risks.	A few hundred drugs, verbatim as drug labels cannot be unreliable.
MedlinePlus health topics	Medical Procedures, types of clinical tests and broad range of both generic and niche health subjects.	Contains general medical topics the other two omit.

Using the three above sources requires the retrieved corpus to be a shared schema, where every single document, contains and is stored as a question, answer, the source that identifies the document and a overall medical subject matter. This ensures all documents are compatible and subject to the retrievers sole and only ranking function rather than having different ranking functions for each corpus. Other potential corpus sources were considered however not pursued due to the existence of synthetic large language model created text and missing named drug label names and another containing poorly structured web scrapped artefacts which obfuscated and would have tainted retrieval. Therefore, choosing to get the raw drug labels from openFDA was used instead, as the data was clean and real.

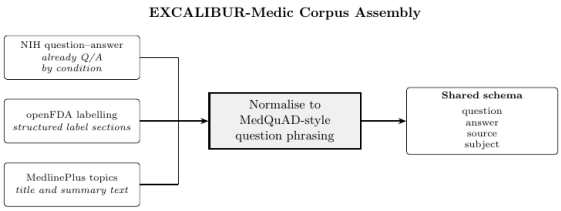


Figure 4.4: How the three input source corpus are assembled into the master single shared schema retrieval corpus.

Additionally, the three input sources must be normalised to the training MedQuAD style questioning, the same exact training corpus that the distillation process is target to, and thus the resultant structure will account for the 4 elements of the schema, being the question, answer, source and subject of all documents in the compiled corpus, the normalisation ensures that every document contains such a structure.

When the EXCALIBUR-Medic model is firstly shipped and deployed to the target edge Pi 5 device, the compiled shared schema retrieval corpus is not shipped with it. Instead, the corpus is built and assembled from it's three constituent data files on first run of the model inference. This ensures there is no duplication of index material that would already exist and be present on device, clamping down on storage that would be duplicated if instead it was shipped, additionally the compiled corpus is timestamped, so future changes can force a rebuild of the updated corpus

rather than statically relying on a strict iteration of the retrieval corpus.

4.4 Ranking the retrieved medical corpus documents

To retrieve the documents from the compiled retrieval source index, documents must be ranked by the BM25 powered sparse retriever. Thereby all documents must be ranked by a sum over certain shared terms, where each is weighted by how discriminating the term at hand is found to be, enforcing saturating with repetition to drop meaningless noise found in source documents where many such unhelpful repetitions occur and finally normalising for the target strict document length boundary. The retriever tunes it's two input parameters against the held-out questioning of the source corpus instead of it's default conventional value to better soak and be grounded in the clinical source corpus.

Using BM25 alone, treats the a users input query question as a bag of undifferentiated words instead of the consistent question, answer, subject matter the compiled corpus has and expects. This means, in practice almost every question involves naming a specific medical condition or drug and asks about a particular attribute of it. This mismatch requires firstly an extraction step to take place to make the mapping from the retriever to the user question possible, where regular pattern recognition is used to determine and extract what the user meant as the subject of the question and the question type to allow for the retrieval ranker to actually understand and reliably locate and retrieve what the user was wanting.

Such a ranking pipeline is visualised in the below figure 4.5. Using the extraction process, it can be seen that a returned document matching the found question type extracted from the users input using regex is boosted in the ranking of a retrieval, thereby meaning it is more likely to be returned as-well as the extracted subject matter of the users query which is more strongly boosted that the extracted question type. Additionally, in index time, the question field rather than the answer field of documents is weighted more strongly as to try and find the most similar corresponding document to what the user queried a question upon, rather than a answer which is answering a similarly worded but different question. Furthermore, broader health topic documents are weighted less as usually they are there to fill gaps in the corpus rather than get in the way or displace specific medical entries and therefore specific documents are ranked more strongly over the more generic clinical documents. Questions where a subject matter cannot be extracted from with full certainty does lose around half of the ranking logic and falls back to vanilla BM25 retrieval performance, however this will usually only occur when the user prompts a question not within the clinical domain and or broken grammatically which would not otherwise be searched upon efficiently or at-all anyway.

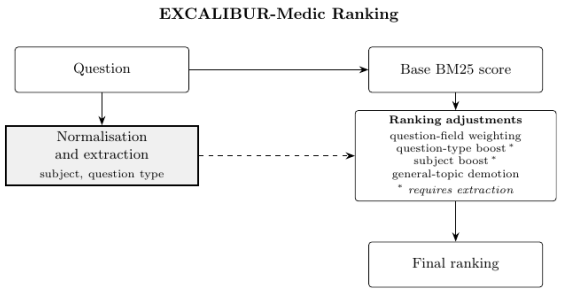


Figure 4.5: How a user question is normalised and ranked through the retriever itself.

There also exists two specific normalisation steps that are enacted before any ranking from the retrieval occurs. The first step is a helper, to better identify user supplied generic names of what verbose drug names and labels are actually identified in the corpus, often as hard to type scientific names that must be mapped to the generic ones users better understand. The other normalisation step is to restore any apostrophes in a user query, for example Alzheimer's disease may be stripped as it contains an apostrophe and when mapped to a specific medical condition name it could be dropped and trimmed, therefore it has to be enforced to exist during tokenisation to match cases in the corpus where it strictly wants to see a apostrophe to discriminate between similar but grammatical different diseases. These two normalisation processes exist to ensure what the user typed in the query can best match the more technical and stricter contents of the retrieval corpus index and is more robust to handle the complexities of human natural language.

Table 4.2: The four weighted adjustments to the retrievers ranking.

Signal	What the design intends	Strength
User query subject matter match	A document about the specific named medical condition or drug should be ranked above documents that just merely mention the user's queried subject matter in passing.	Significant
Question field content	If a document has a title that better suites and states it's subject matter closer to what the user queried instead of the body of the document where it may not closely match, matching the title of a document should rank and count more than matching the body of the text of a document where it may not excitability match.	Strong
Question type existence	There could exist situations where many documents cover the same target subject matter, therefore documents that actually answer said topic should be ranked above those that do not contain answers and are not of a question type.	Tie-breaker only
General healthcare topics	More generic, broader consumer health documents help to fill gaps in the corpus and should not disrupt nor displace specific documents that explicit answer and mention a given disease or subject that will answer the specific questioning better and more reliability than falling back on generic factual content.	Small demotion
Word Saturation and length normalisation bounding	Long documents will be penalised more than shorter documents that more conventionally answer the users question, additionally the breadth of a document match is chosen over extensive repetition of a match to better reward the breadth rather than arbitrary count of key words.	Below retrieval defaults

These four ranking adjustments seen in 4.2 contribute to the final ranking retrieval of a document, when passed through these adjustments represent a more robust domain tuned method of retrieval on a collected, compiled medical corpus that is structured in questions and answers pairs and usually a subject matter determination that aides greatly in retrieving the closest possible match to what the user query was asking rather than using a vanilla BM25 ranking protocol that does not take in these domain specific adjustments in mind.

4.5 The retrievers decision layer

With the consequence of a model not knowing fully the boundary of it's own factual medical knowledge, prompting the model to determine whether it should answer the users medical query goes from a scenario where a question can be measured in answer-ability to one where it cannot be. Instead, the retriever makes the decision on whether or not the answer should be answered, by managing how measured any retrieved source documents relate to what the user queried, to make

the decision whether or not to answer, five such relationships are tested and shown in order in the below table 4.3.

Table 4.3: The five guards, in firing order. Each exists because of an observed failure; thresholds are given in Chapter 5.

Guard	Measures	Failure it prevents	Outcome
Score floor	Check to see if the best retrieved document has score above a minimum threshold.	Unrelated questions from matching something in the source corpus. It must be set quite low as BM25 scales terms in regards to rarity which means common conditions could score below rare irrelevance if not guarded for properly.	Refuse to answer
Question coverage	How much of the user question is weighted by term informativeness and expressiveness and how much a retrieved document can account for.	Typos matching a similar but completely different condition, while missing the only discriminating term that mattered.	Refuse to answer
Corpus Facet gap	If the source corpus actually holds the user aspect queried about.	Questions that pertain to things that the source retrieval corpus does not actually cover and thus prevents the model from inventing the content that does not actually exist at the moment in the corpus.	Offer the answer or refuse
Query disambiguation	Determines whether the user question name something broader than what is actuality indexed in the corpus.	The model returning a general broad answer to one that requires an specific clinical answer, presented as though it were the answer to the users specific questioning.	Offer to answer
Question topic match	Just how much the retrieved source document own subject matter and material exists and appears in the actual prompted question.	The model from performing bare keyword query matching a long and verbose medical document by accident and returning the unrelated specifics with full confidence not actually matching what the user intended while behaving like it actually answered the users queried question.	Refuse to answer

The score floor guard threshold is set deliberately low and coverage proper, of how much the retrieved source documents can actually cover what the user is querying does most of the real work at the decision layer. Furthermore, constraining the floor guard to an absolute score threshold would reject user queries that enquire about common medical conditions whilst allowing for questions about rare irrelevances as a BM25 scores based on how rare a user query's terminology are, not with how relevant the document actually is to the user source, thus setting the threshold to a lower minimum threshold. Additionally, the process of coverage is designed to be scale-free, as it asks what makeup and proportion of the source user question is accounted for by retrieved documents, and is a decision gate layer that can more meaningfully be set rather than floor guard itself.

This also means that the process of refusal is last resort rather than the default process. As a corpus that cannot answer the user question as what was broadly asked could instead offer the narrower

retrieved answerable document entries that it does possess instead. This offer of dis-ambiguity allows the choice of answering that would otherwise be so declined, providing useful and honest choices at more specific questioning that straight up refusal with no option of choice.

The corpus facet gap guard prevents reliability failures, as it ensures that that retrieved symptoms and conditions have to appear in the source corpus documents and that it cannot make up unreliable new conditions as it has to be grounded in the source medical literature, this ensures questions can only generate reasoning and questioning that is grounded in the medical facts and not make up their own synthetic clinical facts.

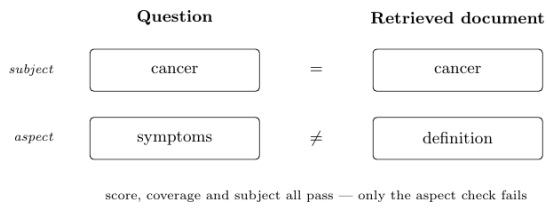


Figure 4.6: The facet guard enforces aspects must be defined in the retrieved source corpus documents

The facet guard figure above 4.6 shows why the two fields, subject and aspect are independently checked. If the only guard was on the subject then it would return a perfect retrieval match, however a mismatch can still occur in the field it would not inspect, the aspect field. Therefore a facet confidence signal that only checks one field can only reliably report on what that singular check examined, and only serves as a assurance about the whole retrieved answer and justified why five separate decision guards exist rather than one composite, as additional guards allow for better servicing and checking of the whole answer as a whole rather than just a sub-component of it.

4.6 The verbatim path

The decision guard establishes whether or not a relevant source document exists to the user supplied input query or not, it does not establish whether or not the answer that the model would generate from it will be faithful and reliable to it. When the cost of such an unreliable answer is highest and most safety critical, the design can offer the verbatim path, that being to remove any and all model generation from it entirely and return the safe passage unchanged and in full, verbatim.

This verbatim path is only ever triggered when the question asked by the user warrants this extra level of securely and faithfully reproducing the answer. Such as in the following six question types, drug dosages, warnings, side effects, interactions, pregnancy and contraindications which are questions that if the model changes unreliable can result in dangerous after-effects and thus must be handled verbatim.

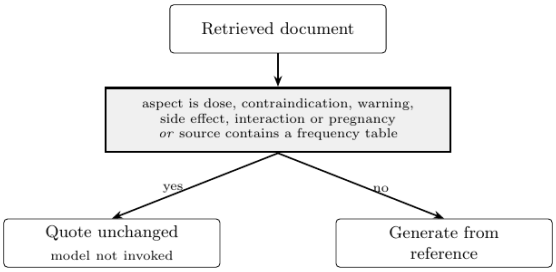


Figure 4.7: The type of questions that are handled verbatim.

Returning a quoted verbatim answer instead of a model generated answer is a trade off, what it will return is verbose, technical source accurate clinical text that will be more difficult to understand and less fluent in it's answering approach. However, it removes any and all possibility for model generation paraphrase error, meaning the model cannot produce incorrect generated text for such safety-critical of user question types.

4.7 Design summary

The multi-component design approach of EXCALIBUR-Medic ensures responsibility is shared and distributed in-order of when components are needed and required. Firstly, compiling and establishing the medical corpus holds all the medical and clinical facts required as such a model of this scale, at one billion parameters cannot hold it on it's own. The retriever works before the model generation is invoked, fetching and checking via scoring and ranking if there are valid retrievable documents related to the users clinical query. At this point, the decision guards determine whether or not a answer can be reliably generated as the model itself cannot be deemed to evaluate its own answer reliability, additionally verbatim pathing can determine and quote safety critical questioning at the expense of fluency and model integration but grounds medical answers in medical facts. Then finally the distilled compressed model, if the decision layers decide the user question can be answered and uses it's distilled behavioural abilities to read a retrieved grounded medical passage and generate and report on it in a consistent, parsable reasoning and answering generation blocks that answers the users query.

This order allows for each component to assess whether or not it can currently act upon and answer the users query and will only act if the presiding step has evidenced that the model generation can go ahead and is safe to generate and respond to the user, rather than inspect integrity after the fact of the current component operation.

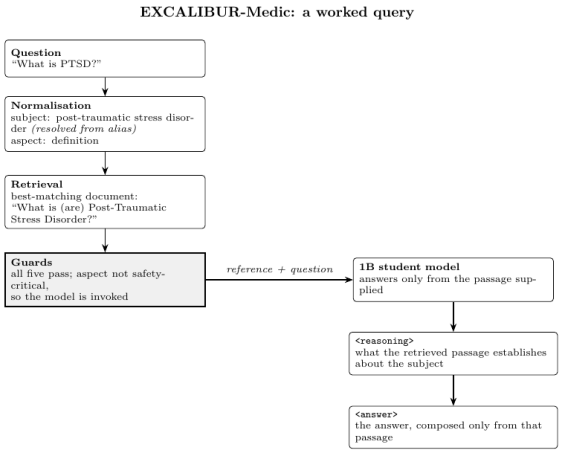


Figure 4.8: A worked query through the EXCALIBUR-Medic design ordered component pipeline

The above figure 4.8 traces a worked user query, with the input query being 'What is PTSD?', from the start to the end of model answer generation. Normalisation is applied to extract what type and subject the user is asking, which in turn allows for the retriever to find an accurate and best matching ground source, from there the retrieval goes through the safeguards and passes, with the question not safety critical enough to warrant the verbatim quotation path, this results in model generation, where the distilled model is handed the retrieved source corpus best-matched reference and the original user query and returns and generates both reasoning and answering blocks in the structured form and thereby returns to standard output it's generated answer response to the user's query.

Table 4.4: Summary of how the system determines when a decision is taken.

System Decision	**Taken by**
Which source corpus document is relevant	The retriever, from the user supplied query
Whether to answer the user clinical query at all	The safe-guards, from the retrieval scores
Whether the EXCALIBUR-Medic model is invoked at all	The verbatim path for safety-critical questions, from extracted aspect and source
How the answer query reads and parses	The model itself, from the supplied and retrieved passage as-well as the original user query

Table 4.4 conveys this decision hierarchy process, ensuring that firstly the choice of a relevant document from retrieval is established before the decision to answer the users query answer at all, this then determines whether or not the EXCALIBUR-Medic model is invoked at all, is it safety critical and requires the verbatim path, and finally the model itself which determines how the answer is parsed and read and presented as generated reasoning and answering to fulfil the user request at answering their given clinical query.

Chapter 5

Implementation

5.1 The three main implementing artefacts of EXCALIBUR-Medic

Implementation of EXCALIBUR-Medic is spread across three major and distinct artefacts, that split implementation function into training, inference and benchmarking separately, allowing for much easier maintainability and deployment.

The first and where training, distillation and supervision occurs is the juypter notebook entitled ***excalibur.ipynb***. This notebook file itself builds the model from end-to-end, runs on the development master machine rather than the target Pi 5 device, and with that machines sufficient memory and compute can apply the building of said model and generate the output quantised model image artefact at around seven hundred megabytes, ready for deploying to the target edge pi 5 device. It is also the only artefact out of the three which is not transferred nor deployed to the Pi 5 device as it is intended only to be run on the development machine as nothing is needed within the notebook at inference time.

Inference is otherwise handled by one singular python 3 source file, titled ***excalibur_inference.py***, this inference script is what will be deployed to the system alongside the model image and the third benchmark artefact. This singular python binary handles all inference logic, assembling the medical corpus, the sparse retriever, safety-guards, and verbatim pathing, as-well as the inference server proper to the generation model.

Finally, the third artefact is the shell script ***benchmark_pi.sh***, this script is deployed with the inference script runtime and measures and evaluates how the model handles token throughput, memory utilisation and thermal behaviour over continuous sustained load and how much energy does a single query draw of the target Pi 5 device.

Table 5.1: The three major implementation artefacts and their purpose, plus whether or not the artefact is deployed on-device on the Pi 5 edge target.

Artefact Name	Role	On device
`excalibur.ipynb`	Builds the model	No
`excalibur_inference.py`	Runs the inference system	Yes
`benchmark_pi.sh`	Measures and benchmarks inference on the Pi 5	Yes

One caveat of splitting the three artefacts apart and separate, is that the notebook has to import the runtime inference script instead of duplicating in order to ensure all the constants involved in inference such as the parameters supplied to the retriever, safeguard threshold values and corpus source path only exist in one place rather be duplicated in two places.

5.2 Building, training and exporting the EXCALIBUR-Medic model

The excalibur notebook training runs exclusively on Apple Silicon arm64 utilising the MLX machine learning platform rather than the multiplatform tensorflow or pytorch. This was chosen as it allows the MacBook Pro M3 Pro (18 GB) based development machine to share one single memory pool between the CPU and GPU thanks to that architectures unified memory, this then allows both models, student and teacher to reside comfortably within the 18 gigabyte memory footprint. The larger teacher model is Llama3-OpenBioLLM-8B, loaded into the notebook at 4-bit precision. The smaller student model is Llama-3.2-1B-Instruct, which is loaded into the notebook at bf16 rather than the 4 bit precision of the teacher model. The choice for bf16 was picked as quantising the student model on both the training and export process would compound the error twice over and as such the export path process applies its own quantisation near the end of the notebook. Both models are downloaded and loaded into the notebook via the hugging face python API. Also, the notebook is ordered in a linear sequence, ran and executed once from a clean juypter kernel, and every generated output artefact that the notebook produces is written to a single directory within the codebase that the excalibur runtime inference script later reads.

The MedQuAD medical corpus is split and handled before anything else in the notebook, where the corpus is grouped by clinical subject type rather than by row index. As the corpus contains many question types per specific condition, a arbitrarily random split of the corpus data would cause *'what causes'* X condition in the training set and *'what are the symptoms of'* X in the test set, causing the resulting evaluation process to measure memorisation of a specific medical condition rather than clinical generalisation to unseen ones, thus enforcing a split on the subject matter in the corpus prevents conditions appearing on both splitting sets and allows for actual measurement and evaluation of model generalisation performance rather than memorisation performance.

The largest block in the notebook is the chain-of-thought distillation generation. 6,000 MedQuAD question and answer pairs are given to the medical teacher model as input to produce and generate text for each pair, both a reasoning passage and an answer derived from the supplied MedQuAD reference. This process takes several hours, averaging approximately seven hours to finish the input 6,000 on the development hardware, and can be resumable when needed. These finished generated samples are output and appended to the codebase as they are produced during the code cell, and a restart of the notebook reads what currently has been processed and finished and continues, ensuring that this process can be interrupted and easily resumed rather than a design where it can only be ran once and if an error or if the system goes to sleep the whole process would need to be done again.

Every training example contains and pairs a bare clinical question along with the medical teacher models tagged generated output. Both the reasoning and answering pairs generated from the teacher model is concatenated into one single completion object, as-well as the loss masked so

only the completion structure is supervised. This means that the smaller student model is trained to produce both the reasoning block and the answer given a input clinical question, and to never reproduce the given input clinical question itself. Reasoning is supervised as ordinary generated tokens rather than be treated specially, which makes the chain-of-thought distillation process a data-formatting decision rather than a fundamental change to the training objective of the student model.

```
# The teacher's two blocks, reasoning and answering, become one tagged completion.
completion = f"<reasoning>\n{reasoning}\n</reasoning>\n<answer>\n{answer}\n</answer>"

ids, prompt_len = render(question, completion) # query template + teacher completion

# Supervise on completion only, never train the model to predict the question, only the
    completion derived from the teacher distillation
mask[b, prompt_len - 1 : len(ids) - 1] = 1.0
loss = (cross_entropy(logits, tokens[:, 1:], reduction="none") * mask).sum() / ntoks
```

Listing 5.1: An example of a chain-of-thought training processed pair. Condensed from the excalibur notebook's data-preparation and batching functions.

Next, training attaches LoRA to a frozen student model. The configuration of the lora is concise enough to state in full and in the following code block.

```
LORA_CFG = {"rank": 16, "scale": 20.0, "dropout": 0.05} # hyperparams for lora adaption
LORA_LAYERS = 16 # all blocks of the student Llama-3.2-1B model

student.freeze() # freeze the student model
linear_to_lora_layers(student, LORA_LAYERS, LORA_CFG) # adaption layers onto frozen
    student
print_trainable_parameters(student) # what the lora trainable parameters of the student
    model will be
```

Listing 5.2: Attaching the LoRA to the student model. Condensed from the configuration and training cells within the excalibur notebook.

The notebook adapts sixteen blocks at a rank rate of 16, which roughly trains around 0.3 per cent of the total student model's parameters, a scale that ensures the optimiser state of the model can be stored comfortably in the notebook in megabytes rather than several gigabytes. This scale of LoRA allows the training of the student to sit and run alongside the resident medical teacher model at runtime, and it is also why several training stages could be attempted and then promptly discard within the same memory budget. Val loss during the training process is recorded throughout. At the final step of training is the adapter that will be saved alongside regression between this final step and from the step where the best observed val loss has been observed, thereby showing how close the training in it's final step has diverged from the best observed state if such a state has been observed differently from final training value. During last run, this value was observed to be 0.0006 nats pitted against a threshold of 0.05 which would have rolled back to the minimum value instead.

The export section of the notebook follows once distillation training of the student has completed,

and is what will take the trained adapter into the deployed artefact that the edge Pi 5 device runs. Firstly the student adapter is fused into the base weights, then the fused student model is converted via llama-cpp to a 16 bit GGUF image, then an importance matrix is computed by measuring over a calibration corpus drawn from the stored training data in the previous section of the model, the resultant calibrated importance matrix is then used by the model to be quantised against.

```
run([BIN/"llama-imatrix", "-m", F16, "-f", CALIB, "-o", IMAT,
    "-c", "512", "--chunks", "400"], "imatrix") # computes the imatrix
run([BIN/"llama-quantize", "--imatrix", IMAT, F16, OUT, "Q4_K_M"], "quantize") # quantizes
     based off the computed imatrix above
```

Listing 5.3: Importance-matrix quantisation step of the export process.

Every step of the exportation process is guarded by a staleness check, which ensures rebuilding only occurs when the output of the current export step either absent or metadata deemed older than what is expected from where it derives from so a partial rerun of the excalibur notebook does not repeat hours of work in regards to exportation. The export stage then culminates into a final step that verifies the quantised model file by asking it a medical question and checking the response from the file, this verification judges not just if the export quantised model file merely exists but if it is actually functioning as expected and can return both a reasoning and answering response from the input text question.

Additionally, once after the export section of the model, the notebook also contains many figures pertaining to the evaluation of the training and distillation process of the model as-well it's compression, of which are reproduced on each run of the notebook, thus the notebook can reproduce itself visually the evaluation metrics of the notebook's objectives that will be shown and supplied in the evaluation section.

5.3 The deployed edge runtime inference system for EXCALIBUR-Medic

The runtime script that holds the inference logic for EXCALIBUR-Medic is a single Python language module depending on nothing outside the standard library of Python 3. As such there exists no additional numerical library nor no embedding model to suite. While there exists a network http call, this is just a call to the llama-cpp runtime server running locally on the edge Pi 5 device therefore nothing leaves the device during inference. The retrievers medical index is implemented as a set of dictionary mapping terms to corresponding typed arrays of document regular identifiers, the scoring of the retriever is arithmetic and the only process that is external within the inference script is the aforementioned llama-cpp inference server. Choosing to forgo many external dependencies was a deliberate choice to allow for the relatively simple requirement of solely the llama-cpp runtime and deployed generated files from the notebook, namely the compressed quantised GGUF image, the inference script itself and the benchmarking script.

On first use of the inference script the retrieval index is built once and cached for future use. As building of the index only takes an extra secound or two and the corpus that comprise the index rarely change, the cache is the normal path for such a inference as the cache carries both

a timestamp and a data-stamp identifying the corpus makeup of what it was built from and will only rebuild itself when the stamp no longer matches i.e. when the fundamental corpus changes or if the cache itself gets corrupted. This bypasses a scenario where an earlier version of the index is used report on a much newer corpus whilst still generating plausible results from the wrong out-dated data, such a failure wouldn't be identified without careful tracking of the cached index.

Within the deployed inference file, a singular python function decides on what actually happens to a input user query question. The function returns a tuple of four data elements, where the presence or absence of messages in the first element of the deciding method determines whether or not the EXCALIBUR-Medic model is invoked at all.

```
# returns (messages, grounded, options, verbatim)

return refuse(user) # 1. decline, the model is told that
                                    # no source exists and not to guess
return ([], False, options, None) # 2. offer narrower medical questions
return ([], True, [], quoted_passage) # 3. quote the source unchanged, no model, the
    verbatim path
return ([system, user_with_reference], # 4. answer from the found retrieved reference
       True, [], None)
```

Listing 5.4: The four potential exits from `build_messages`, gathered from separate points in the function. An empty message list means the model is never called, i.e. not relevant enough clinically or no source can answer it or verbatim quoting rather than generation.

Only two of the four potential exits carry no messages, so no request is actually issues to the model and thus the model contributes nothing to the response of these two exists. Exit 1 does actually invoke the model, but instructs the model to generate text that explains to the user that no references were found so the question cannot be reliably answered, this is expected and is the process of abstention behaviour as trained within the excalibur notebook. Routing the refusal phrasing through the EXCALIBUR-Medic model itself rather from a scripted templates is what allows the model to fit the user question that was asked, and further enhance the refusal output by actually acknowledging the contents of the user message and why it was refused instead of a blanket and unhelpful refusal.

What the deciding safeguards actually threshold is one particular query quality, coverage. By thresholding coverage most importantly, a user query is scored on how much of the question query the retrievers best matching retrieved document can account for, weighting each term of the question by how rare it occurs across the retrieval index corpus, thus for a clinical query matching symptoms carry less of a weight and count less for a specific matching condition term such as *hypothyroidism* which is weighed more heavily and of course counts for a great deal more over matching symptoms.

```
max_idf = max(ix.idf.values())

for w in query_terms:
    weight = ix.idf.get(w, max_idf) # unknown term token infers as maximally informative
        for the retriever, due to typo catching
```

```
    total += weight
    if w in doc_terms:
        matched += weight

coverage = matched / total # coverage gets computed
```

Listing 5.5: IDF-weighted coverage application: how much of the user input question the retrieved document can accounts for. Empty-index and zero-total safe-guards omitted for legability.

A term that the retrieval index corpus has never been exposed to nor seen is treated as being maximally informative rather than being ignored by the coverage lookup. This allows for often misspelt condition names to drag down coverage rather than being complete ignored and going unnoticed, this allows for the typo to therefore be caught, if the retriever weightlessly treated unknown terms instead, then it would never catch the typo and would lead to many silent failures as medical conditions are often typo prone due to their overtly verbose and scientific naming conventions.

The models text generation process runs against the serving llama-server subprocess spawned by the inference script. Allowing for the streaming of generated text, token by token so that model output appears just as it is produced on the screen rather than wait until after a complete response, this allows the user to see the model reasoning and answering being generated in real time rather than wait even longer for the model to actually display its already generated tokens and better shows that the model is actually working and generating a response. The server port of the llama-cpp runtime is always the same reused configured port rather than replacing it, this allows for the benchmark system to continually and deterministically make responses to the runtime server.

```
ap.add_argument("--no-rag", action="store_true",
                help="ablation: disable retrieval") # disables rag during model runtime

ap.add_argument("--allow-ungrounded", dest="strict", action="store_false",
                help="let the model answer with no reference document found") # allows for
                    the no reference found generations during runtime, allows for unsafe
                    output generation
ap.set_defaults(strict=True)
```

Listing 5.6: The two command line arguments for the inference script. Both exist only so that the unaided and no-retrieval conditions can be measured and evaulated against a normal run.

The runtime inference script is supplied with two optional command line arguments for use in measuring the models inference. The first argument, no rag, disables retrieval completely to measure inference with rag retrieval totally disabled. The other, allows for the answering of questions that otherwise have no grounded reference. Neither of these two arguments are intended for the end user to use, and only exist to measure how well the model performs when both features are disabled, allowing for grounding comparisons when permutations are enabled and disabled.

Table 5.2: Everything required on the Pi 5 edge device for interactive questioning and answering inference.

EXCALIBUR-Medic Component	**File**	**Size**
Runtime Script	`excalibur_inference.py`	100 KB
Exported EXCALIBUR Model	`excalibur-medic-1b-Q4_K_M.gguf`	770 MB
Corpus sources for retrieval index	`medquad.csv`, `drug_labels.csv`, `medlineplus.csv`	29 MB
Alias tables for retrieval	`drug_aliases.json`, `topic_aliases.json`	$<$ 1 MB
Inference server	`llama-server`, built from `llama.cpp` on the device for maximum hardware optimisation	—
Python Interpreter	Python 3, standard library only, no need for third party pips	—
	Total data transferred	**$\approx$ 800 MB**

These are the only files required for the entire deployment of inference. As such there exists no additional package installation set, no requirement for python virtual environment setup and the compiled medical retrieval corpus and generated index is created in the first run of the inference rather than be deployed with the rest of the files as there is no need to duplicate data of which the deployed source file already contain and can generate on device. As there exists quite a few files to be transferred and deployed to the pi 5, the codebase comes with a deploy to pi shell script which transfers these files automatically via ssh.

5.4 The EXCALIBUR-Medic Model edge benchmarking process

The model benchmark runs only on device on the Pi 5 edge device, and has two distinct modes.

The first mode and the default mode of the benchmark when passed without any arguments, is to first sweep thread counts starting at one and all the way to four, the max core count of the pi 5 device against model generation and prompt throughput, recording peak resident model memory for two configurations, one with a 1024 token context size plus a full precision cache, and the secound being the deployed 2048 token context size with a quantised cache instead of full precision, it also counts how long the retrieval index build process takes to complete and perform an initial query. Stages of end to end retrieval and generation turn occur for both variants of the two memory configurations, with the initial token load captured, followed by additional, responses to varying queries, by the end average token per secounds and runtime and average memory utilisation is all captured and reported back both in standard output and in the working directory as a text file. Generation and prompt throughput is computed and benchmarked separately in tokens per secound, retrieval latency is also measured in milliseconds to determine what is the retrieval latency. The peak resident memory for each memory variant is given and recorded in megabytes.

The actual throughput is retrieved and taken from the llama.cpp token counter rather than directly counting streamed frames to the benchmark, as a frame is not always guaranteed to carry a generated token. Additionally, peak memory is taken and read from directly from Linux kernel

high-water mark itself so to not miss any potential transient spikes in the memory that sampling using more high level software timing could miss.

```
gen=$(jnum "$resp" predicted_per_second) # llama-cpp's server runtime own token counter
prm=$(jnum "$resp" prompt_per_second)

rss=$(awk '/VmHWM/{print $2}' "/proc/$pid/status") # linux kernel high-water mark, more
    senstivite to memory fluctuations.
```

Listing 5.7: Token throughput, runtime and peak memory benchmarking. Drawn from varying thread-sweep and memory sections respectively;

The secound mode of the model benchmark, which is accessed by supplying the –energy command line argument to the benchmark shell script solely focuses on benchmarking and measuring power. This benchmark requires the computation of the total board power that the Pi 5 power management chip exposes, where during benchmarking is sampled per secound by a shell background process while the actual excalibur model inference workload runs. This then allows for the following four energy quantities to be established and recorded, first an idle floor power draw before any model server infers, second an idle floor with the server loaded and resident in memory, then a benchmark for computing the energy consumed per user query across twenty diverse questions, and lastly a forty minute sustained load stress test reporting or whether or not the model thermal throttles the pi 5 edge device, and if so, when the throughput decays and at what measured temperature. The forty-minute stress test is important as a device that is fast for the first question inference and extremely slow after half an hour of clinical question and answering inference is not.

```
vcgencmd pmic_read_adc | awk '
  /current\(/ { amp[rail] = value } # parsing of rail and value omitted, is set as current
       computed power
  /volt\(/ { vlt[rail] = value } # voltage there-after
  END { for (r in amp) if (r in vlt) w += amp[r] * vlt[r]; printf "%.4f", w }' # formatted
       power output wattage
```

Listing 5.8: Raspberry Pi 5 Board power: Wattage the sum of $V \times I$ over every rail that the PMIC Pi 5 chip reports.

Although this is not raw wall power measurement, the energy benchmark uses the Pi 5's hardware power management chip power rails, ignores USB and peripheral power draw noise, and when factoring in both an established idle power draw and sustained load during question inference and establishing the differential between them both, that particular limitation largely cancels out and the power measuring is accurate.

Chapter 6

Evaluation

6.1 Model distillation and retrieval notebook results

The following evaluation figures report back what the EXCALIBUR-Medic model answers correctly, and much more importantly, which component of the excalibur model is responsible. All figures in this section are generated and reproduced by the notebook itself, and not only are the figures created and output to a plots sub-directory within the excalibur codebase, these figures are also embedded and can be accessed and viewed within the excalibur notebook file itself.

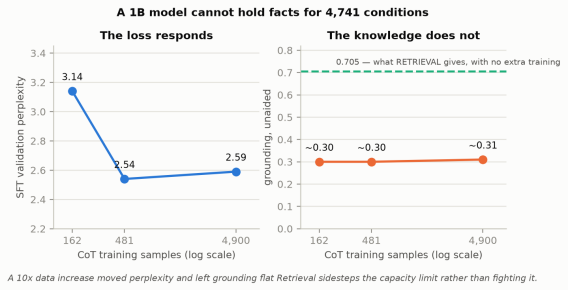


Figure 6.1: The one billion parameter student model capacity ceiling

The first result of the evaluation is a clear negative one and strongly determined and justified the overall architecture of the system. Figure 6.1 shows an overall tenfold increase in model training data, ground performance remained flat ~ 0.30 while validation perplexity continued to fall. This shows that the student model was learning the register of the domain and not its actual contents. This behaviour was predicted in the background section, wherein a small language model of this parameter scale cannot encode the several thousand hyper domain specific clinical conditions and facts requires, and that training on additional domain data does not fundamentally change such a capacity constraint. Therefore training on more medical corpus data was not an available route to better clinical answering from the model.

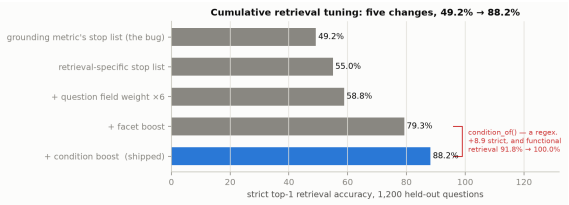


Figure 6.2: Retrieval ladder of the model, showing how each tuning adjustments added to the overall retrieval accuracy on the 1,200 held out MedQuAD question pairs.

Instead, if model capacity cannot be raised itself, then the alternative route to improving model retrieval performance is to supply the medical knowledge at inference time. This retrieval was tuned across five cumulative changes, and measured on a set of 1,200 held out corpus questions as seen in fig 6.2. Ensuring proper stop list correction took top-1 k accuracy from 49.2 to 55.0 per cent, applying weighting to the question field further improved accuracy to 58.8 percent, incorporating the domain facet boosts reached 79.3 percent, and the condition boost, which boost both the subject question matter along with tunes to the query saturation and length parameters reached the maximum 88.2 percent strict accuracy and a full 100 percent on functional accuracy. The largest such single contribution to the accuracy came therefore from extracting via regular expression the medical condition name from the query question and is worth stating plainly, that the biggest retrieval gain in EXCALIBUR-Medic was not model powered itself but raw regex information supplication.

When measured against the end-to-end metric, adding in retrieval to the model takes answer grounding from 0.314 to 0.705, close to a perfect oracle ceiling of 0.716 when supplying the correct medical reference directly. Supplying with retrieval therefore recovers almost all of the possible available headroom in regards to grounding accuracy.

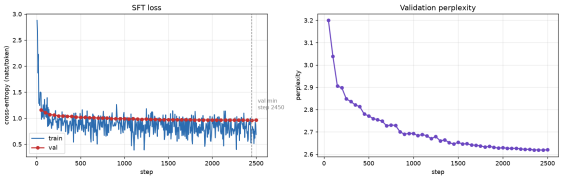


Figure 6.3: Supervised Fine Tuning (SFT) training loss and validation perplexity

When measuring the supervised fine-tuning loss from the chain of thought teacher generations, it ran for the full 2,500 training steps with val loss capturing, and the measured curve converges cleanly and without much showing of instability, both the loss and validation perplexity is shown in figure 6.3.

Validation perplexity from SFT fell from a value of 3.14 when 162 training samples in to a value of 2.54 at 481 samples in, however it then rose to a value 2.59 at 4,9000 training samples and rose a bit more to 2.62 once the behaviour corpus logic was added in. Thus more training data did not uniformly improve the SFT training metric. While adding in the behavioural examples cost the training 0.03 in validation perplexity, it took model formal compliance from 80 to 100 per cent on the held out MedQuAD corpus questions. This means that the model did become marginally

worse at predicting its own training distribution but it did become substantially better at the things the system actually requires to be good at, such as format compliance.

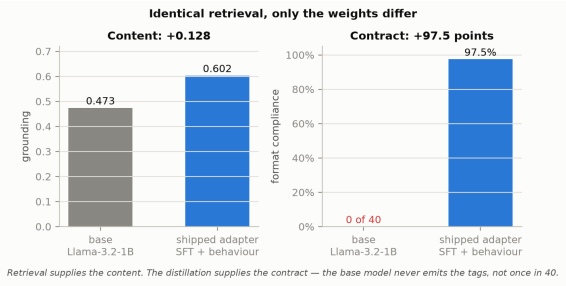


Figure 6.4: Base student model versus distilled student model grounding and inference compliance, showing that distillation does indeed improve over the base model.

This begs the question. If retrieval is the component that is actually supplying the model quality, could the model distillation process be unnecessary? Figure 6.4 explores and tests this thesis directly, it uses forty held out medical corpus questions and records both the grounding and format compliance against the base model and the SFT distilled model. This figure shows the base model scores 0.473 in medical answering grounding and a flat 0 percent for the model format compliance. On the other hand, the shipped SFT adapter model scores a better 0.602 for grounding and a 97.5 per cent format compliance.

The SFT grounding gain of 0.128 is genuine and worth having as it shows more reliable generated answering. The format compliance on the other hand is categorical, as the base model never generates the required reasoning and answer tags, not once in the forty held out question attempts from the corpus, and as the EXCALIBUR-Medic system was designed and requires on both tags, with the styled model output that the user will see, along with the verbatim pathing all assume a well-formed model response that the base model just cannot provide. And since retrieval cannot supply this functionality, distillation clearly has it's purposes and the system can not perform without its careful and learnt reasoning and answer generation tagging.

What can be evidenced, is that both components, retrieval and distillation is important and required in-order for the system to work correctly. As retrieval supplies the content and concrete grounding whereas the distillation supplies the actual model contract, neither can be optional and both have to be used. And while additional corpus training does not improve the model content, it does in fact improve the model form.

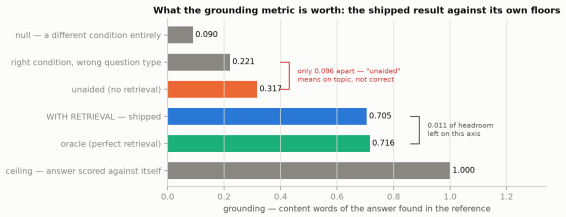


Figure 6.5: Grounding ladder, what it is worth and the shipped adapted model against it's own floors

It is worth stating that most of the figures rest upon the metric of grounding. That being what proportion of a generated answer tokens appear in it's medical reference, which ensures measurement of the model performance is measured in regards to topicality. The above figure 6.5 details exactly what a grounding score means and the ceilings between the unaided baseline model, the adapted model with retrieval and the maximum possible perfect retrieve in the corpus, compared to when the answer is scored against itself. The reason perfect retrieval is ~0.7 and not closer to 1.0 is due to the fact the model generates text that paraphrase from a retrieval and does not just copy and paste the answer text itself, thus meaning it can never reached a perfect 1.0 as the model actually has to generate the reasoning and answering blocks, but grounding shows how closely and reliable the output text is to what is actually sourced in the reference clinical corpus.

What these figures do not measure is the real world target Pi 5 model performance, both compute, memory, and energy measurements, of which will be documented and evidenced in the next section.

6.2 Raspberry pi 5 on-device deployment measurements

The table below contains the general benchmark metrics from running benchmark_pi.sh on the remote Raspberry Pi 5 2 GB edge device, the general benchmark focuses primarily on compute and memory measuring. Computing how well the EXCALIBUR-Medic model runs both in time and in space results, when supplied varying degrees of example clinical questions and on different thread permutations and memory configurations.

Table 6.1: Compute and memory measurements on the Raspberry Pi 5 using benchmark_pi.sh (2 GB, headless, Raspberry Pi OS Lite).

Measurement	**Generation**	**Prompt**
Throughput, 1 thread	9.6 tok/s	21.9 tok/s
Throughput, 2 threads	**14.6 tok/s**	44.3 tok/s
Throughput, 3 threads	13.7 tok/s	63.4 tok/s
Throughput, 4 threads	12.7 tok/s	**77.7 tok/s**
RAM available	1815 MB of 2010 MB	
Peak RSS, 1024 ctx / f16 KV	1636 MB — 943 MB free	
Peak RSS, 2048 ctx / q8_0 KV (deployed)	**1638 MB — 929 MB free**	
BM25 index build / query (ARM)	3.3 s / 7.8 ms, 19,411 docs	
End-to-end retrieval turn	44.4 s at `-t 2 -tb 4`; 49.2 s at `-t 4`	

Power instead was measured passing the –energy onto the benchmark_pi.sh script, two independent runs of the energy benchmark is given, so that reproducibility could also be assessed against two distinct runs.

Table 6.2: Power and thermal behaviour, two independent runs measured from benchmark_pi.sh –energy.

Measurement	**Run 1**	**Run 2**
Model resident, idle	1.51 W	1.50 W
Delta over a bare board	−0.04 W	−0.01 W
Sustained generation	5.38 W	5.41 W
Median answer, marginal	62.0 J	61.4 J
Average draw at 20 queries/hour	1.85 W	1.84 W
Throttling across 40 minutes	none	none

It can be evidenced from the general device performance benchmark that the task of generation and processing the prompt want different thread counts, with generation being more memory bandwidth bound rather than compute, as dropping from four threads to two only degrades compute performance by around fourteen percent. While, on the other hand prompt processing is much more compute bound and nearly doubles in performance with the thread bump. Double the context length costed the model an additional 2 megabytes of memory once the key value cache has been quantised, which effectively makes the larger context size free and leaves a comfortable 929 MB free in memory, plus keeping the model resident in memory did not cost anything measurable as evidenced by the above table 6.1. What can be gathered from the energy benchmark is that general user clinical queries are cheap, drawing marginal wattage per answered query, thermal throttling does not occur during sustained model generation, meaning that the model is not draining every last bit of compute from the Pi 5 and still has headroom in terms of both compute and memory as evidenced from the general benchmark that averages around 900 megabytes still free for the Linux operating system.



Figure 6.6: Interactive inline per query stats

Alternatively, using the stats keyword interactively in the model inference can provide per user query model performance metrics. See figure 6.6 above, when given the example prompt of *what is hypertension?*, the retrieval, token speed for both generation and prompting. cpu utilisation, memory utilisation and thermal and energy is given below the user query for when the user has enabled inline stats mode.

Chapter 7

Discussion

The question presented and poised by the thesis title was whether both distillation and retrieval together can enable and support medical questioning inference at one billion parameters on the target deployable Raspberry Pi 5 edge device, which is such a hardware target that existing surveyed literature rules out due to the shear memory and compute limitations of the 2 GB Pi 5 target. Turns out, as evidenced in this report, they can. With the resultant EXCALIBUR-Medic model providing an answering inference environment within 1,638 MB of system addressable memory, and at a 14.6 token generated per secound and at 5 watts of power. Where the model, under a heavy 40 minute sustained load clinical answer inference benchmark showed no signs of thermal throttling, and when retrieval is added to the model itself, it takes unaided medical answer grounding from 0.314 to 0.705 very close to the maximum achievable ceiling headroom of 0.716. Therefore, a system built around the shape as required from the available memory constraints can both be built and run reasonably quickly on the device with the generated reliable sourced-backed text generating reasoning and answering response blocks to the user query, quicker than most can read and at a peak energy that is extremely sustainable.

The contribution however of both retrieval and distillation cannot be said to be symmetric, as what both contribute to the model is not similar. As measuring the distilled model from the base model grounding alone only shows a slight increase of 0.128 however, when measuring model behaviour, such as it's format compliance it rises from complete zero to 97.5 percent allowing for the complex interactions with the retriever and it's subcomponents. It can be said that retrieval alone supplies the factual content that could not have been stored in the one billion parameter alone and distillation supplies the contract for delivering it, deciding on the medical tone and style to use as learnt from the teacher model. This research is helpful for anyone building something similar to what was developed in this work, as training a small language model solely on hyper-specific domain facts is a mostly wasted effort as the inherit parameter capacity of such small language models is not there to hold them. But, instead training it via distilled output from how the medical teacher treated the clinical discipline is not, and using this distillation, a model can learn how to best structure, handle tone, semantics and style when fed such clinical facts whereas retrieval can never gain nor supply such vital meta knowledge at any medical corpus size.

What is worth mentioning is that two of the largest improvements to the systems performance were also the cheapest to implement which was unexpected. As the biggest retrieval performance

gain came only from a handful of regex targetted at extracting the exact condition name if one exists in a user queries using regex matching alone. Additionally, one of the biggest gains to the model answer quality came from reducing the sampling temperature down to zero which allowed for the recovery of 7 out of 8 expected drug names in a scenario that looked like it needed the entire clinical training corpus regenerated. There was around 15 hours of alternative training strategies that produced results worse than the SFT chain of thought distillation process and thus were not shipped. This presented the case that cheaper options are often worth checking first as they are easy to implement and remove if they don't work and problems that initially resemble like they are capacity bound may in fact turn out to be more decoding bound instead.

Rather than learning safe guarding, the model architecture itself is devoted to safety first. As the safeguards are presented in the retrieval stage and decided upon before any model generation as it was found that the model itself cannot be a reliable judge on whether or not it should actually go about answering the user medical query or not.

Due to the design and implementation of EXCALIBUR-Medic, there exists certain caveats and scenarios of which the good performance will deviate. For example, the largest retrieval accuracy gain, as previously mentioned, came from the source medical corpus being expressive and regular enough that a few regex patterns can determine a subject matter from almost every domain relevant user query, i.e. the user actually naming the subject on what they what a answered generation from. If the compiled domain was instead comprised of unstructured prose, free-form questions it will lose the subject boost capabilities and also the domain derived facet-gap guard that requires such accessibly extractable query text, and thus losing the biggest retrieval accuracy gain that the system contains and one out of the five safety guard mechanisms. Although the system architecture would survive such a different domain, much of what grants good performance would not survive. The current main downside to the model and it's system of components is that one out of every ten realistic medical query still goes unanswered, and such questions go unanswered due to factual gaps in the source corpus of the model rather than fundamental failures in either the retrieval process or distilled powered medical reasoning and answering generation.

Chapter 8

Conclusion

This work opened with a question, can distillation and retrieval enable on-device clinical inference on 2 GB edge devices? The completed EXCALIBUR-Medic model shows that the two components can. A 1 billion parameter student model distilled from a 8 billion parameter medical teacher model, was compressed and quantised to a model that is 770 megabytes in size and on the disk, and when paired with a sparse BM25 index retriever, over around 19,411 clinical documents compiled as it's indexed corpus ground source, answers clinical questions within a memory footprint of around 1,638 megabytes leaving a few hundred megabytes free for the operating system at a throughput of around 14.6 tokens per secound at an energy draw of roughly five watts, this performance can sustain a heavy load for 40 minutes without thermal throttling the Pi 5 edge device and requires absolutely no network at the point of inference.

The most important findings of the EXCALIBUR-Medic work is not just that it runs, but how the model work divides. Although distillation alone only increases clinical source grounding by rate of 0.128 instead, it more helpfully and importantly learns the structural medical reasoning and answering behaviour from a flat zero rate to a near perfect rate of 97.5 per cent in model format compliance. Retrieval of-course supplies the model with the factual knowledgable content, but the distillation process supplies the compatibility by which that factual content is deliverable. Neither can work sufficiently alone, and the noted consequence of training at such a small scale as this work does, is that the effort for training belongs to capturing how an domain output orderliness should be instead of a raw stream of factual facts that the parameters at this scale cannot hold sufficiently. This ensures the correct division of labour for both major component (retrieval and distillation), to allow for such robust and safe interactive medical answering inference at this scale comfortably and reliably on-device on a 2 GB Raspberry Pi 5 edge device.

8.1 Limitations & Future work

Currently, the inference treats every user query independently and does not remember the context of the previous query, this is one of the reasons why the model itself is never referred to a chat model, as it currently only has the context for the current user query. When the EXCALIBUR-Medic model is asked the following question, What is PTSD? it answers as expected however, if the user answers nextly, how do you get it? It has no subject matter to extract as it does not keep the previous PTSD query in context, and thus both the subject booster and clinical facet-gap safe

guards fail and the question is very likely to be refused. If the model instead carried the context from the previous turn subject text over, then it would allow for the resolution of a large class of human natural follow up questions at a moderate cost to context. It was not implemented in the final state of EXCALIBUR-Medic due to time constraints rather than the difficulty of such a two pass context memory.

A further consideration for future work, is the fact that most if not all of the medical corpus originates from US data and is thus US-authored and US biased. This is a problem as if the model generates crisis and or emergency services numbers it will generate the US variants of said numbers rather than any regional equivalents the user might be expecting. This is more apparent in the domain of drug labels, which are retrieved from the American FDA source rather than the European standards, so potential retrievals from these sources may generate different drug approvals, the names of the drugs itself may be different and drug warnings can differ from where the device region legally says. The US training data was chosen as it was found to be more easily accessible and better documented than other regional versions, and thus future work could be to localise the corpus to European or British variant, which would better ground generations in what is legally and regionally true when the model is used in these regions.

Furthermore, the model itself has not yet been reviewed by a licensed clinician. By arranging a structured, comprehensive medical review of the EXCALIBURMedic carried out by a licensed clinician, the model could better be evaluated rather than currently by the correct medical literature grounding, but instead , more robustly, if the model demonstrably gives the right clinically approved answer itself, by implementing such a future work, the models evaluation metrics would be more domain relevant, clinically accurate and less generalised to typical language model evaluation techniques.

Lastly, a current limitation of the EXCALIBUR-Medic work is that the excalibur training notebook currently only works on Apple Silicon hardware and is tied to Apple's MLX framework rather than a cross platform framework such as PyTorch or TensorFlow. This restricts heavily the hardware that can reproduce the training results. Instead, a future work could be to refactor and re-implement the training in PyTorch so that the pipeline could be run under Nvidia, AMD, Intel acceleration and or CPU only acceleration workflows. However, the deployed artefact GGUF distilled and compressed model is currently hardware-agnostic and can be run on any such llama.cpp supporting platform, the actual process, i.e. the excalibur notebook that produces said model is not, and thus re-implementing the notebook model pipeline into the better supported PyTorch platform will remove the remaining bottleneck in independent reproduction of the training, distillation adaption and compression results.

Appendix A

Appendix A: Source code, reproduction artefacts and user manual

A.1 GitHub Repository for Excalibur

The complete source code for EXCALIBUR-Medic can be found in the following GitHub repository:

Excalibur Medic GitHub Repo

It contains the 3 main implementing artefacts, the excalibur notebook, inference medical answering runtime script and the benchmark as-well as complementing scripts. It does not however contain the trained model weights, the resultant quantised image model file nor the distilled generated data however due to GitHub's file size limits and ensuring reproduction can happen cleanly. These files are instead retrievable and accessible in the below google drive section.

A.2 Optional excalibur training artefacts

The files that are not included in the GitHub repository instead can be downloaded in the below google drive link:

Training Artefacts

Each stage removes a training stage, all are optional as the excalibur notebook will produce them unaided although lengthy when the notebook is run end to end.

Table A.1: Optional reproduction artefacts and the stage each removes.

File	Size	Saves	Place in
`cot_full.jsonl`	13 MB	11.65 h	`artifacts/synthetic/`
`sft_adapter.safetensors`	43 MB	46 min	`artifacts/checkpoints/`
`excalibur-medic-1b-Q4_K_M.gguf`	807 MB	20 min	`artifacts/`

The 13 MB file matters the most in regards to shear training time, as just directly copying it into the codebase saves around 11 and a half hours of training runtime making reproduction more time feasible, it is also the only step that requires the use of the 8B teacher medical model resident.

A.3 Setting up the training environment

Two machines are required; a Mac with an ARM CPU and around 18 GB of unified memory, executes the excalibur notebook and builds the model. A Raspberry Pi 5 2 GB running Raspberry Pi OS Lite runs it. On the Mac build the llama.cpp runtime first as the notebook requires it for calling is converted and quantiser during export.

```
git clone https://github.com/ggml-org/llama.cpp ~/llama.cpp
cmake -S ~/llama.cpp -B ~/llama.cpp/build -DCMAKE_BUILD_TYPE=Release
cmake --build ~/llama.cpp/build -j 10

python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python -m pip install -e ~/llama.cpp/gguf-py
.venv/bin/python -m ipykernel install --user --name excalibur \
    --display-name "EXCALIBUR (.venv)"
```

A.4 Building the excalibur model

Start by opening the excalibur.ipynb notebook, and make sure that juypter has loaded a virtual environment python kernel of 3.14.6, and run it end to end. If you want to avoid the lengthy chain of thought generation, you can place the cot_full.jsonl in artifacts/synthetic. After running the notebook, it will have generated all model artefacts ready for deployment.

A.5 Deploying to the Pi 5

Firstly, on the remote Pi 5 device, build the llama.cpp runtime with native ARM instructions.

```
sudo apt install -y cmake g++ git python3
git clone https://github.com/ggml-org/llama.cpp ~/llama.cpp
cmake -S ~/llama.cpp -B ~/llama.cpp/build -DCMAKE_BUILD_TYPE=Release -DGGML_NATIVE=ON
cmake --build ~/llama.cpp/build -j4 --target llama-server llama-cli
```

Then from the mac, run the deploy to pi helper script.

```
ssh pi@raspberrypi.local 'mkdir -p ~/excalibur/artifacts' # change hostname and local
↪  addressing if needed

./deploy_to_pi.sh                              # default host
HOST=pi@other.local ./deploy_to_pi.sh          # a different board
```

It will prompt for the pi password in order to transfer the files via ssh.

A.6 Inference and asking questions on the Pi 5

```
cd ~/excalibur

python3 excalibur_inference.py                      # interactive
python3 excalibur_inference.py --lite               # prompt only
python3 excalibur_inference.py -q "What is the dose of metformin?"
python3 excalibur_inference.py --stats              # per-answer telemetry
```

The first run of the inference script will take a few extra secounds to build the retrieval index and start running the llama.cpp runtime on the pi. Running it in interactive mode is the best place to start as it gives you some sample example user query's to type and see the model return in real time it's reasoning and answering to said questions.

A.7 Benchmarking on the Pi 5

```
./benchmark_pi.sh            # threads, peak memory, BM25 on ARM, end-to-end
./benchmark_pi.sh --energy   # ~50 min: idle floor, joules per query, thermal soak
```

As the energy benchmark requires the use of the Pi 5 power-management chip, it will not work on any other single board computer.

Full excalibur setup and installation instructions as-well as troubleshooting tips are given in the codebase file, INSTALLATION.md.

A.8 Video demonstration of EXCALIBUR-Medic

Below is a short video demonstration of the EXCALIBUR-Medic model in action, answering user medical queries and generating both reasoning and answering blocks to what the user queried originally and a run of the general benchmark.

Video Demonstration

Bibliography

Abacha, A. B. and Demner-Fushman, D. (2019). A Question-Entailment Approach to Question Answering. *BMC Bioinformatics*, 20(1):511. arXiv:1901.08079 [cs.CL].

Abdin, M., Aneja, J., Awadalla, H., Awadallah, A., Awan, A. A., Bach, N., Bahree, A., Bakhtiari, A., Bao, J., Behl, H., Benhaim, A., Bilenko, M., Bjorck, J., Bubeck, S., Cai, M., Cai, Q., Chaudhary, V., Chen, D., Chen, D., Chen, W., Chen, Y.-C., Chen, Y.-L., Cheng, H., Chopra, P., Dai, X., Dixon, M., Eldan, R., Fragoso, V., Gao, J., Gao, M., Gao, M., Garg, A., Del Giorno, A., Goswami, A., Gunasekar, S., Haider, E., Hao, J., Hewett, R. J., Hu, W., Huynh, J., Iter, D., Jacobs, S. A., Javaheripi, M., Jin, X., Karampatziakis, N., Kauffmann, P., Khademi, M., Kim, D., Kim, Y. J., Kurilenko, L., Lee, J. R., Lee, Y. T., Li, Y., Li, Y., Liang, C., Liden, L., Lin, X., Lin, Z., Liu, C., Liu, L., Liu, M., Liu, W., Liu, X., Luo, C., Madan, P., Mahmoudzadeh, A., Majercak, D., Mazzola, M., Mendes, C. C. T., Mitra, A., Modi, H., Nguyen, A., Norick, B., Patra, B., Perez-Becker, D., Portet, T., Pryzant, R., Qin, H., Radmilac, M., Ren, L., de Rosa, G., Rosset, C., Roy, S., Ruwase, O., Saarikivi, O., Saied, A., Salim, A., Santacroce, M., Shah, S., Shang, N., Sharma, H., Shen, Y., Shukla, S., Song, X., Tanaka, M., Tupini, A., Vaddamanu, P., Wang, C., Wang, G., Wang, L., Wang, S., Wang, X., Wang, Y., Ward, R., Wen, W., Witte, P., Wu, H., Wu, X., Wyatt, M., Xiao, B., Xu, C., Xu, J., Xu, W., Xue, J., Yadav, S., Yang, F., Yang, J., Yang, Y., Yang, Z., Yu, D., Yuan, L., Zhang, C., Zhang, C., Zhang, J., Zhang, L. L., Zhang, Y., Zhang, Y., Zhang, Y., and Zhou, X. (2024). Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone.

Allal, L. B., Lozhkov, A., Bakouch, E., Blázquez, G. M., Penedo, G., Tunstall, L., Marafioti, A., Kydlíček, H., Lajarín, A. P., Srivastav, V., Lochner, J., Fahlgren, C., Nguyen, X.-S., Fourrier, C., Burtenshaw, B., Larcher, H., Zhao, H., Zakka, C., Morlon, M., Raffel, C., von Werra, L., and Wolf, T. (2025). Smollm2: When smol goes big – data-centric training of a small language model.

Chen, J., Cai, Z., Ji, K., Wang, X., Liu, W., Wang, R., Hou, J., and Wang, B. (2024). HuatuoGPT-o1, Towards Medical Complex Reasoning with LLMs. arXiv:2412.18925 [cs.CL].

Chen, L., Zaharia, M., and Zou, J. (2023a). FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. arXiv:2305.05176 [cs.LG].

Chen, Z., Cano, A. H., Romanou, A., Bonnet, A., Matoba, K., Salvi, F., Pagliardini, M., Fan, S., Köpf, A., Mohtashami, A., Sallinen, A., Sakhaeirad, A., Swamy, V., Krawczuk, I., Bayazit, D., Marmet, A., Montariol, S., Hartley, M.-A., Jaggi, M., and Bosselut, A. (2023b). MEDITRON-70B: Scaling Medical Pretraining for Large Language Models. arXiv:2311.16079 [cs.CL].

Chow, C. (1970). On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46.

Del Fiol, G., Workman, T. E., and Gorman, P. N. (2014). Clinical questions raised by clinicians at the point of care: a systematic review. *JAMA internal medicine*, 174(5):710–718.

Dettmers, T., Lewis, M., Belkada, Y., and Zettlemoyer, L. (2022). LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. arXiv:2208.07339 [cs.LG].

Dettmers, T. and Zettlemoyer, L. (2023). The case for 4-bit precision: k-bit Inference Scaling Laws. arXiv:2212.09720 [cs.LG].

ECJ (2020). Data Protection Commissioner v Facebook Ireland Limited and Maximillian Schrems.

EU (2016). REGULATION (EU) 2016/ 679 OF THE EUROPEAN PARLIAMENT AND OF THE COUNCIL - of 27 April 2016 - on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/ 46/ EC (General Data Protection Regulation). *Official Journal of the European Union*.

EU (2024). Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence and amending Regulations (EC) No 300/2008, (EU) No 167/2013, (EU) No 168/2013, (EU) 2018/858, (EU) 2018/1139 and (EU) 2019/2144 and Directives 2014/90/EU, (EU) 2016/797 and (EU) 2020/1828 (Artificial Intelligence Act) (Text with EEA relevance). Legislative Body: CONSIL, EP.

Geng, S., Josifoski, M., Peyrard, M., and West, R. (2023). Grammar-Constrained Decoding for Structured NLP Tasks without Finetuning. In Bouamor, H., Pino, J., and Bali, K., editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10932–10952, Singapore. Association for Computational Linguistics.

ggml org (2026). ggml-org/llama.cpp. original-date: 2023-03-10T18:58:00Z.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A., Korenev, A., Hinsvark, A., Rao, A., Zhang, A., Rodriguez, A., Gregerson, A., Spataru, A., Roziere, B., Biron, B., Tang, B., Chern, B., Caucheteux, C., Nayak, C., Bi, C., Marra, C., McConnell, C., Keller, C., Touret, C., Wu, C., Wong, C., Ferrer, C. C., Nikolaidis, C., Allonsius, D., Song, D., Pintz, D., Livshits, D., Wyatt, D., Esiobu, D., Choudhary, D., Mahajan, D., Garcia-Olano, D., Perino, D., Hupkes, D., Lakomkin, E., AlBadawy, E., Lobanova, E., Dinan, E., Smith, E. M., Radenovic, F., Guzmán, F., Zhang, F., Synnaeve, G., Lee, G., Anderson, G. L., Thattai, G., Nail, G., Mialon, G., Pang, G., Cucurell, G., Nguyen, H., Korevaar, H., Xu, H., Touvron, H., Zarov, I., Ibarra, I. A., Kloumann, I., Misra, I., Evtimov, I., Zhang, J., Copet, J., Lee, J., Geffert, J., Vranes, J., Park, J., Mahadeokar, J., Shah, J., Linde, J. v. d., Billock, J., Hong, J., Lee, J., Fu, J., Chi, J., Huang, J., Liu, J., Wang, J., Yu, J., Bitton, J., Spisak, J., Park, J., Rocca, J., Johnstun, J., Saxe, J., Jia, J., Alwala, K. V., Prasad, K., Upasani, K., Plawiak, K., Li, K., Heafield, K., Stone, K., El-Arini, K., Iyer, K., Malik, K., Chiu, K., Bhalla, K., Lakhotia, K., Rantala-Yeary, L., Maaten, L. v. d., Chen, L., Tan, L., Jenkins, L., Martin, L., Madaan, L., Malo, L., Blecher, L., Landzaat, L., Oliveira, L. d., Muzzi, M., Pasupuleti, M., Singh, M., Paluri, M., Kardas, M., Tsimpoukelli, M., Oldham, M., Rita, M., Pavlova, M., Kambadur, M., Lewis, M., Si, M., Singh, M. K., Hassan, M., Goyal, N., Torabi,

N., Bashlykov, N., Bogoychev, N., Chatterji, N., Zhang, N., Duchenne, O., Çelebi, O., Alrassy, P., Zhang, P., Li, P., Vasic, P., Weng, P., Bhargava, P., Dubal, P., Krishnan, P., Koura, P. S., Xu, P., He, Q., Dong, Q., Srinivasan, R., Ganapathy, R., Calderer, R., Cabral, R. S., Stojnic, R., Raileanu, R., Maheswari, R., Girdhar, R., Patel, R., Sauvestre, R., Polidoro, R., Sumbaly, R., Taylor, R., Silva, R., Hou, R., Wang, R., Hosseini, S., Chennabasappa, S., Singh, S., Bell, S., Kim, S. S., Edunov, S., Nie, S., Narang, S., Raparthy, S., Shen, S., Wan, S., Bhosale, S., Zhang, S., Vandenhende, S., Batra, S., Whitman, S., Sootla, S., Collot, S., Gururangan, S., Borodinsky, S., Herman, T., Fowler, T., Sheasha, T., Georgiou, T., Scialom, T., Speckbacher, T., Mihaylov, T., Xiao, T., Karn, U., Goswami, V., Gupta, V., Ramanathan, V., Kerkez, V., Gonguet, V., Do, V., Vogeti, V., Albiero, V., Petrovic, V., Chu, W., Xiong, W., Fu, W., Meers, W., Martinet, X., Wang, X., Wang, X., Tan, X. E., Xia, X., Xie, X., Jia, X., Wang, X., Goldschlag, Y., Gaur, Y., Babaei, Y., Wen, Y., Song, Y., Zhang, Y., Li, Y., Mao, Y., Coudert, Z. D., Yan, Z., Chen, Z., Papakipos, Z., Singh, A., Srivastava, A., Jain, A., Kelsey, A., Shajnfeld, A., Gangidi, A., Victoria, A., Goldstand, A., Menon, A., Sharma, A., Boesenberg, A., Baevski, A., Feinstein, A., Kallet, A., Sangani, A., Teo, A., Yunus, A., Lupu, A., Alvarado, A., Caples, A., Gu, A., Ho, A., Poulton, A., Ryan, A., Ramchandani, A., Dong, A., Franco, A., Goyal, A., Saraf, A., Chowdhury, A., Gabriel, A., Bharambe, A., Eisenman, A., Yazdan, A., James, B., Maurer, B., Leonhardi, B., Huang, B., Loyd, B., Paola, B. D., Paranjape, B., Liu, B., Wu, B., Ni, B., Hancock, B., Wasti, B., Spence, B., Stojkovic, B., Gamido, B., Montalvo, B., Parker, C., Burton, C., Mejia, C., Liu, C., Wang, C., Kim, C., Zhou, C., Hu, C., Chu, C.-H., Cai, C., Tindal, C., Feichtenhofer, C., Gao, C., Civin, D., Beaty, D., Kreymer, D., Li, D., Adkins, D., Xu, D., Testuggine, D., David, D., Parikh, D., Liskovich, D., Foss, D., Wang, D., Le, D., Holland, D., Dowling, E., Jamil, E., Montgomery, E., Presani, E., Hahn, E., Wood, E., Le, E.-T., Brinkman, E., Arcaute, E., Dunbar, E., Smothers, E., Sun, F., Kreuk, F., Tian, F., Kokkinos, F., Ozgenel, F., Caggioni, F., Kanayet, F., Seide, F., Florez, G. M., Schwarz, G., Badeer, G., Swee, G., Halpern, G., Herman, G., Sizov, G., Guangyi, Zhang, Lakshminarayanan, G., Inan, H., Shojanazeri, H., Zou, H., Wang, H., Zha, H., Habeeb, H., Rudolph, H., Suk, H., Aspegren, H., Goldman, H., Zhan, H., Damlaj, I., Molybog, I., Tufanov, I., Leontiadis, I., Veliche, I.-E., Gat, I., Weissman, J., Geboski, J., Kohli, J., Lam, J., Asher, J., Gaya, J.-B., Marcus, J., Tang, J., Chan, J., Zhen, J., Reizenstein, J., Teboul, J., Zhong, J., Jin, J., Yang, J., Cummings, J., Carvill, J., Shepard, J., McPhie, J., Torres, J., Ginsburg, J., Wang, J., Wu, K., U, K. H., Saxena, K., Khandelwal, K., Zand, K., Matosich, K., Veeraraghavan, K., Michelena, K., Li, K., Jagadeesh, K., Huang, K., Chawla, K., Huang, K., Chen, L., Garg, L., A, L., Silva, L., Bell, L., Zhang, L., Guo, L., Yu, L., Moshkovich, L., Wehrstedt, L., Khabsa, M., Avalani, M., Bhatt, M., Mankus, M., Hasson, M., Lennie, M., Reso, M., Groshev, M., Naumov, M., Lathi, M., Keneally, M., Liu, M., Seltzer, M. L., Valko, M., Restrepo, M., Patel, M., Vyatskov, M., Samvelyan, M., Clark, M., Macey, M., Wang, M., Hermoso, M. J., Metanat, M., Rastegari, M., Bansal, M., Santhanam, N., Parks, N., White, N., Bawa, N., Singhal, N., Egebo, N., Usunier, N., Mehta, N., Laptev, N. P., Dong, N., Cheng, N., Chernoguz, O., Hart, O., Salpekar, O., Kalinli, O., Kent, P., Parekh, P., Saab, P., Balaji, P., Rittner, P., Bontrager, P., Roux, P., Dollar, P., Zvyagina, P., Ratanchandani, P., Yuvraj, P., Liang, Q., Alao, R., Rodriguez, R., Ayub, R., Murthy, R., Nayani, R., Mitra, R., Parthasarathy, R., Li, R., Hogan, R., Battey, R., Wang, R., Howes, R., Rinott, R., Mehta, S., Siby, S., Bondu,

S. J., Datta, S., Chugh, S., Hunt, S., Dhillon, S., Sidorov, S., Pan, S., Mahajan, S., Verma, S., Yamamoto, S., Ramaswamy, S., Lindsay, S., Lindsay, S., Feng, S., Lin, S., Zha, S. C., Patil, S., Shankar, S., Zhang, S., Zhang, S., Wang, S., Agarwal, S., Sajuyigbe, S., Chintala, S., Max, S., Chen, S., Kehoe, S., Satterfield, S., Govindaprasad, S., Gupta, S., Deng, S., Cho, S., Virk, S., Subramanian, S., Choudhury, S., Goldman, S., Remez, T., Glaser, T., Best, T., Koehler, T., Robinson, T., Li, T., Zhang, T., Matthews, T., Chou, T., Shaked, T., Vontimitta, V., Ajayi, V., Montanez, V., Mohan, V., Kumar, V. S., Mangla, V., Ionescu, V., Poenaru, V., Mihailescu, V. T., Ivanov, V., Li, W., Wang, W., Jiang, W., Bouaziz, W., Constable, W., Tang, X., Wu, X., Wang, X., Wu, X., Gao, X., Kleinman, Y., Chen, Y., Hu, Y., Jia, Y., Qi, Y., Li, Y., Zhang, Y., Zhang, Y., Adi, Y., Nam, Y., Yu, Wang, Zhao, Y., Hao, Y., Qian, Y., Li, Y., He, Y., Rait, Z., DeVito, Z., Rosnbrick, Z., Wen, Z., Yang, Z., Zhao, Z., and Ma, Z. (2024). The Llama 3 Herd of Models. arXiv:2407.21783 [cs.AI].

Han, T., Adams, L. C., Papaioannou, J.-M., Grundmann, P., Oberhauser, T., Figueroa, A., Löser, A., Truhn, D., and Bressem, K. K. (2025). MedAlpaca – An Open-Source Collection of Medical Conversational AI Models and Training Data. arXiv:2304.08247 [cs.CL].

Hinton, G., Vinyals, O., and Dean, J. (2015). Distilling the Knowledge in a Neural Network. arXiv:1503.02531 [stat.ML].

Ho, N., Schmid, L., and Yun, S.-Y. (2023). Large Language Models Are Reasoning Teachers. arXiv:2212.10071 [cs.CL].

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Casas, D. d. L., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., Driessche, G. v. d., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Rae, J. W., Vinyals, O., and Sifre, L. (2022). Training Compute-Optimal Large Language Models.

Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., Laroussilhe, Q. d., Gesmundo, A., Attariyan, M., and Gelly, S. (2019). Parameter-Efficient Transfer Learning for NLP. arXiv:1902.00751 [cs.LG].

Hsieh, C.-Y., Li, C.-L., Yeh, C.-k., Nakhost, H., Fujii, Y., Ratner, A., Krishna, R., Lee, C.-Y., and Pfister, T. (2023). Distilling Step-by-Step! Outperforming Larger Language Models with Less Training Data and Smaller Model Sizes. In Rogers, A., Boyd-Graber, J., and Okazaki, N., editors, *Findings of the Association for Computational Linguistics: ACL 2023*, pages 8003–8017, Toronto, Canada. Association for Computational Linguistics.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685 [cs.CL].

Hu, S., Tu, Y., Han, X., He, C., Cui, G., Long, X., Zheng, Z., Fang, Y., Huang, Y., Zhao, W., Zhang, X., Thai, Z. L., Zhang, K., Wang, C., Yao, Y., Zhao, C., Zhou, J., Cai, J., Zhai, Z., Ding, N., Jia, C., Zeng, G., Li, D., Liu, Z., and Sun, M. (2024). Minicpm: Unveiling the potential of small language models with scalable training strategies.

ITU (2025). Measuring digital development: Facts and Figures 2025.

Jeong, D. P., Mani, P., Garg, S., Lipton, Z. C., and Oberst, M. (2025). The Limited Impact of Medical Adaptation of Large Language and Vision-Language Models. arXiv:2411.08870 [cs.CL].

Ji, X. (2026). SLM (small language models) - a Ji-Xiang Collection.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Chen, D., Dai, W., Chan, H. S., Madotto, A., and Fung, P. (2023). Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12):1–38. arXiv:2202.03629 [cs.CL].

Jin, D., Pan, E., Oufattole, N., Weng, W.-H., Fang, H., and Szolovits, P. (2020). What Disease does this Patient Have? A Large-scale Open Domain Question Answering Dataset from Medical Exams. arXiv:2009.13081 [cs.CL].

Jin, Q., Dhingra, B., Liu, Z., Cohen, W., and Lu, X. (2019). PubMedQA: A Dataset for Biomedical Research Question Answering. In Inui, K., Jiang, J., Ng, V., and Wan, X., editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2567–2577, Hong Kong, China. Association for Computational Linguistics.

Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., Schiefer, N., Hatfield-Dodds, Z., DasSarma, N., Tran-Johnson, E., Johnston, S., El-Showk, S., Jones, A., Elhage, N., Hume, T., Chen, A., Bai, Y., Bowman, S., Fort, S., Ganguli, D., Hernandez, D., Jacobson, J., Kernion, J., Kravec, S., Lovitt, L., Ndousse, K., Olsson, C., Ringer, S., Amodei, D., Brown, T., Clark, J., Joseph, N., Mann, B., McCandlish, S., Olah, C., and Kaplan, J. (2022). Language Models (Mostly) Know What They Know. arXiv:2207.05221 [cs.CL].

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. (2020). Scaling Laws for Neural Language Models.

Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., and Yih, W.-t. (2020). Dense Passage Retrieval for Open-Domain Question Answering. arXiv:2004.04906 [cs.CL].

Labrak, Y., Bazoge, A., Morin, E., Gourraud, P.-A., Rouvier, M., and Dufour, R. (2024). BioMistral: A Collection of Open-Source Pretrained Large Language Models for Medical Domains. arXiv:2402.10373 [cs.CL].

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2021). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. arXiv:2005.11401 [cs.CL].

Lobo, E., Agarwal, C., and Lakkaraju, H. (2025). On the Impact of Fine-Tuning on Chain-of-Thought Reasoning. arXiv:2411.15382 [cs.CL].

Magister, L. C., Mallinson, J., Adamek, J., Malmi, E., and Severyn, A. (2023). Teaching Small Language Models to Reason. arXiv:2212.08410 [cs.CL].

Mehta, S. (2025). OVision A raspberry Pi powered portable low cost medical device framework for cancer diagnosis. *Scientific Reports*, 15(1):7124.

Miller, N., Lacroix, E.-M., and Backus, J. E. B. (2000). MEDLINEplus: building and maintaining the National Library of Medicine's consumer health Web service. *Bulletin of the Medical Library Association*, 88(1):11–17.

Nobari, A. H., Alimohammadi, K., ArjomandBigdeli, A., Srivastava, A., Ahmed, F., and Azizan, N. (2025). Activation-Informed Merging of Large Language Models. arXiv:2502.02421 [cs.CL] version: 1.

Nori, H., King, N., McKinney, S. M., Carignan, D., and Horvitz, E. (2023). Capabilities of GPT-4 on Medical Challenge Problems. arXiv:2303.13375 [cs.CL].

Organisation mondiale de la santé, editor (2016). *World health statistics 2016: monitoring health*

for the SDGs sustainable development goals. World health organization, Genève.

Pal, A., Umapathi, L. K., and Sankarasubbu, M. (2022). MedMCQA : A Large-scale Multi-Subject Multi-Choice Dataset for Medical domain Question Answering. arXiv:2203.14371 [cs.CL].

Pal, A., Umapathi, L. K., and Sankarasubbu, M. (2023). Med-HALT: Medical Domain Hallucination Test for Large Language Models. In Jiang, J., Reitter, D., and Deng, S., editors, *Proceedings of the 27th Conference on Computational Natural Language Learning (CoNLL)*, pages 314–334, Singapore. Association for Computational Linguistics.

PiHut (2026). Raspberry Pi 5.

Qwen, :, Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Tang, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Wan, Y., Liu, Y., Cui, Z., Zhang, Z., and Qiu, Z. (2025). Qwen2.5 technical report.

Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., and Finn, C. (2024). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. arXiv:2305.18290 [cs.LG].

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 21(140):1–67.

Rashkin, H., Nikolaev, V., Lamm, M., Aroyo, L., Collins, M., Das, D., Petrov, S., Tomar, G. S., Turc, I., and Reitter, D. (2023). Measuring Attribution in Natural Language Generation Models. *Computational Linguistics*, 49(4):777–840.

Robertson, S. and Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends® in Information Retrieval*, 4(1-2):1–174.

Sanh, V., Debut, L., Chaumond, J., and Wolf, T. (2020). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108 [cs.CL].

Schaeffer, R., Miranda, B., and Koyejo, S. (2023). Are Emergent Abilities of Large Language Models a Mirage? arXiv:2304.15004 [cs.AI].

Sim, S. Z. Y. and Chen, T. (2025). Critique of impure reason: Unveiling the reasoning behaviour of medical large language models. *eLife*, 14:e106187.

Singhal, K., Azizi, S., Tu, T., Mahdavi, S. S., Wei, J., Chung, H. W., Scales, N., Tanwani, A., Cole-Lewis, H., Pfohl, S., Payne, P., Seneviratne, M., Gamble, P., Kelly, C., Babiker, A., Schärli, N., Chowdhery, A., Mansfield, P., Demner-Fushman, D., Agüera y Arcas, B., Webster, D., Corrado, G. S., Matias, Y., Chou, K., Gottweis, J., Tomasev, N., Liu, Y., Rajkomar, A., Barral, J., Semturs, C., Karthikesalingam, A., and Natarajan, V. (2023). Large language models encode clinical knowledge. *Nature*, 620(7972):172–180.

SONIA, P. (2020). Development of Low-Cost Point-of-Care Technologies for Cervical Cancer Prevention Based on a Single-Board Computer. *IEEE Journal of Translational Engineering in Health and Medicine*, 8:4300210.

Sumit (2023). Generative Retrieval for End-to-End Search Systems. Section: posts.

Team, G., Riviere, M., Pathak, S., Sessa, P. G., Hardin, C., Bhupatiraju, S., Hussenot, L., Mesnard,

T., Shahriari, B., Ramé, A., Ferret, J., Liu, P., Tafti, P., Friesen, A., Casbon, M., Ramos, S., Kumar, R., Lan, C. L., Jerome, S., Tsitsulin, A., Vieillard, N., Stanczyk, P., Girgin, S., Momchev, N., Hoffman, M., Thakoor, S., Grill, J.-B., Neyshabur, B., Bachem, O., Walton, A., Severyn, A., Parrish, A., Ahmad, A., Hutchison, A., Abdagic, A., Carl, A., Shen, A., Brock, A., Coenen, A., Laforge, A., Paterson, A., Bastian, B., Piot, B., Wu, B., Royal, B., Chen, C., Kumar, C., Perry, C., Welty, C., Choquette-Choo, C. A., Sinopalnikov, D., Weinberger, D., Vijaykumar, D., Rogozińska, D., Herbison, D., Bandy, E., Wang, E., Noland, E., Moreira, E., Senter, E., Eltyshev, E., Visin, F., Rasskin, G., Wei, G., Cameron, G., Martins, G., Hashemi, H., Klimczak-Plucińska, H., Batra, H., Dhand, H., Nardini, I., Mein, J., Zhou, J., Svensson, J., Stanway, J., Chan, J., Zhou, J. P., Carrasqueira, J., Iljazi, J., Becker, J., Fernandez, J., van Amersfoort, J., Gordon, J., Lipschultz, J., Newlan, J., Ji, J.-y., Mohamed, K., Badola, K., Black, K., Millican, K., McDonell, K., Nguyen, K., Sodhia, K., Greene, K., Sjoesund, L. L., Usui, L., Sifre, L., Heuermann, L., Lago, L., McNealus, L., Soares, L. B., Kilpatrick, L., Dixon, L., Martins, L., Reid, M., Singh, M., Iverson, M., Görner, M., Velloso, M., Wirth, M., Davidow, M., Miller, M., Rahtz, M., Watson, M., Risdal, M., Kazemi, M., Moynihan, M., Zhang, M., Kahng, M., Park, M., Rahman, M., Khatwani, M., Dao, N., Bardoliwalla, N., Devanathan, N., Dumai, N., Chauhan, N., Wahltinez, O., Botarda, P., Barnes, P., Barham, P., Michel, P., Jin, P., Georgiev, P., Culliton, P., Kuppala, P., Comanescu, R., Merhej, R., Jana, R., Rokni, R. A., Agarwal, R., Mullins, R., Saadat, S., Carthy, S. M., Cogan, S., Perrin, S., Arnold, S. M. R., Krause, S., Dai, S., Garg, S., Sheth, S., Ronstrom, S., Chan, S., Jordan, T., Yu, T., Eccles, T., Hennigan, T., Kocisky, T., Doshi, T., Jain, V., Yadav, V., Meshram, V., Dharmadhikari, V., Barkley, W., Wei, W., Ye, W., Han, W., Kwon, W., Xu, X., Shen, Z., Gong, Z., Wei, Z., Cotruta, V., Kirk, P., Rao, A., Giang, M., Peran, L., Warkentin, T., Collins, E., Barral, J., Ghahramani, Z., Hadsell, R., Sculley, D., Banks, J., Dragan, A., Petrov, S., Vinyals, O., Dean, J., Hassabis, D., Kavukcuoglu, K., Farabet, C., Buchatskaya, E., Borgeaud, S., Fiedel, N., Joulin, A., Kenealy, K., Dadashi, R., and Andreev, A. (2024). Gemma 2: Improving Open Language Models at a Practical Size.

Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., and Gurevych, I. (2021). BEIR: A Heterogenous Benchmark for Zero-shot Evaluation of Information Retrieval Models. arXiv:2104.08663 [cs.IR].

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. (2023). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv:2201.11903 [cs.CL].

Wu, C., Lin, W., Zhang, X., Zhang, Y., Wang, Y., and Xie, W. (2023). PMC-LLaMA: Towards Building Open-source Language Models for Medicine. arXiv:2304.14454 [cs.CL].

Xiong, G. (2024). Improving Retrieval-Augmented Generation in Medicine with Iterative Follow-up Questions.

Xiong, G., Jin, Q., Lu, Z., and Zhang, A. (2024). Benchmarking Retrieval-Augmented Generation for Medicine. arXiv:2402.13178 [cs.CL].

Yin, Z., Sun, Q., Guo, Q., Wu, J., Qiu, X., and Huang, X. (2023). Do Large Language Models Know What They Don't Know? In Rogers, A., Boyd-Graber, J., and Okazaki, N., editors, *Findings of the Association for Computational Linguistics: ACL 2023*, pages 8653–8665, Toronto, Canada. Association for Computational Linguistics.

Zhang, P., Zeng, G., Wang, T., and Lu, W. (2024). TinyLlama: An Open-Source Small Language Model.